A Tutorial in Dynamic Systems Simulation and Modelling With R, ctsem, and lme4: Couples' Affect Dynamics Over Time

Charles C. Driver and Michael Aristodemou
Department of Psychology, University of Zurich

Understanding how people change is a fundamental goal of psychological science. However, translating complex ideas about psychological dynamics into formal models can be challenging without tools that make the modelling assumptions explicit. In this tutorial, we demonstrate components of an iterative workflow for dynamic systems modelling in R: theory is translated into a generative model, the model is simulated with understandable functions, a corresponding statistical model is estimated, model-implied behavior is checked, and the specification is revised, simplified, or extended. We begin with familiar linear models in lme4 and then transition to ctsem, which allows us to estimate models with state-dependent change, random fluctuation distinct from measurement error, covariate effects, interactions, external inputs, and time-varying parameters. These continuous-time modelling concepts are illustrated using a running example of affect dynamics in couples therapy, but the same principles apply to other psychological domains, and many of the modelling principles also apply to discrete-time forms.

Impact Statement

This tutorial shows how psychological researchers can use R and the ctsem package to build and understand dynamic systems models while keeping the modelling assumptions visible. By working from theory to simulations, estimation, checking, and revision, we aim to facilitate the translation of theoretical concepts into empirical models and improve understanding of psychological processes and their interrelations over time.

Keywords: dynamic systems, continuous time, hierarchical modelling, longitudinal data analysis, affect dynamics, ctsem, lme4, state-space models

Dynamic systems theory provides a framework for understanding how psychological constructs evolve over time (van Geert, 2011). To turn conceptual ideas about dynamic systems into insight about mechanisms of change, theory must be translated into empirically testable models, which when combined with data can in turn suggest updates to theory (Borsboom et al., 2021). Yet, this translation from conceptual ideas to testable statistical models can be difficult without tools that make the steps transparent.

This tutorial is designed to help psychological researchers bridge the gap between theoretical concepts in dynamic systems and their practical implementation using statistical modelling. The central contribution is the components of a workflow: translate theory into a generative model, simulate that model with transparent R functions, estimate a corresponding statistical model, check whether the model-implied behavior matches the intended theory and observed data, and then revise, simplify, or extend the specification. We begin with familiar linear mixed-effects models using the R package lme4 (Bates et al., 2015), then gradually move into the dynamic systems and continuous-time domain using the package ctsem (Driver & Voelkle, 2018a).

A strength of `ctsem` is that it can accommodate this full modelling continuum. Starting from basic linear growth models, one can move through discrete- and continuous-time versions of classic frameworks such as the Cross-Lagged

Charles C. Driver
Michael Aristodemou

An earlier version of this manuscript is available as a PsyArXiv preprint at https://osf.io/preprints/psyarxiv/4q9ex_v3. The accompanying OSF project, including manuscript materials and code, is available at https://osf.io/xh6ue/. The examples use simulated data generated for the tutorial. The authors declare that they have no conflicts of interest to disclose. Author roles were classified using the Contributor Role Taxonomy (CRediT; https://credit.niso.org/) as follows: Charles C. Driver: conceptualization, methodology, software, validation, writing, visualization; Michael Aristodemou: formal analysis, writing, visualization

Correspondence concerning this article should be addressed to Charles C. Driver, Department of Psychology, University of Zurich, Binzmühlestrasse 14, Box 28, Zurich, Zurich 8057, Switzerland, Email: charles.driver@psychologie.uzh.ch

Panel Model (Kenny, 2005), the random intercept extension of it (Hamaker et al., 2015), and related forms such as network vector autoregression (Epskamp, 2020). From there, one can extend ctsem to models with considerably more complexity. These extensions include complex variation across individuals, oscillations, and measurement or dynamics parameters that shift over time or in response to moderators and interventions. In this way, ctsem provides a flexible environment for building models up step by step: researchers can begin with simple forms to establish a baseline, then incorporate the dynamics that theory suggests, then perhaps go further with data-driven model and theory development. While this tutorial emphasises the `ctsem` package, and specific syntax and package issues are discussed, the principles demonstrated are general and can be applied to other packages and modelling approaches, as well as to discrete-time forms (also available in `ctsem` but not emphasised here).

Continuous-time representations are particularly valuable for theory-oriented model development, because of the more natural link between continuous-time parameters and typical theoretical interests (Aalen et al., 2016; Driver & Tomasik, 2023; Ryan & Hamaker, 2022). Because many psychological processes — such as emotion regulation, stress responses, symptom fluctuation, learning, physiological regulation, or cognitive fatigue — are thought to evolve continuously rather than only when measured, continuous-time models provide a natural representation. Parameters such as drift (state dependence), diffusion (system noise), and external inputs align much more closely with mechanistic interpretations(Driver, 2025), instead of as artifacts of the measurement schedule as with discrete-time models. Continuous-time models are also useful when observation intervals vary or when the time scale of change is substantively important.

Simulation is a core element in a modelling workflow because it can make the consequences of theoretical assumptions visible before empirical estimation. All simulations in this tutorial are pieced together from simpler R functions, to facilitate a clear link between the model and the simulation. It is of course important to recognise that simulated data will not resemble every empirical complication, but this approach helps understand how model parameters generate trajectories, when different sources of variability have different implications, and how a fitted model can fail even when it explains a large amount of variance. Simulation builds the basis of understanding necessary for applying the principles of dynamic systems modelling to inevitably more complex empirical data, and can also help researchers determine approximate requirements in terms of data density, timing, and reliability, under specific assumptions (however, this is not a tutorial on power analysis or sample size determination).

Boundary Conditions

Continuous-time dynamic systems models are most useful when the research question concerns change over time, when theory implies that current states influence subsequent change, when observation timing matters, or when parameters can be linked to interpretable features of an evolving system. They can be especially valuable for intensive longitudinal data, irregularly spaced observations, intervention or event processes, dyadic and networked systems, and questions about heterogeneity in dynamic processes. They are not automatically preferable, however. Simpler descriptive, mixed-effects, or discrete-time models may be better choices when the research question is not dynamic, when the time scale is not theoretically meaningful, when measurements are too sparse or unreliable to identify the proposed dynamics, or when the parameters of a more complex model would not be substantively interpretable.

This work develops models from simpler to more complex, but model complexity should be calibrated in both directions: A model can be too complex for the data, leading to weak identification, unstable estimates, or estimation failures. A model can also be too simple however: distinct influences may be bundled into a single parameter and then interpreted as if that parameter represented one psychological process. In practice, model choice should triangulate theory, measurement design, observation density, model-implied behavior, diagnostics, and parsimony. `ctsem` can in principle allow the specification of models of arbitrary dimension and complexity, but in practice computational and or data limitations will limit dimensionality and the non-linear filtering approach of `ctsem` will only provide rough approximations to more extreme nonlinear systems. Important empirical complications not fully treated here include missing-not-at-random, non-stationarity, floor or ceiling effects, non-Gaussian outcomes, and many other forms of misspecification.

Scope and Outline

In this tutorial we develop intuition for dynamic systems modelling by building complexity with a running example regarding the impact of couples therapy on affect dynamics over time. The example is grounded in research showing that partners' affective states are dynamically interdependent, becoming synchronized through processes of mutual influence (Butler & Randall, 2013). Such coupling can either amplify or dampen emotional experiences, with implications for individual well-being and relationship stability. Although the examples use affect in dyads, the modelling approaches apply wherever theory concerns change over time, coupling among processes, heterogeneity, and external inputs. The tutorial is organised as follows:

Section: Modelling Univariate Growth introduces univariate change models in `lme4`. Linear Change describes steady

linear growth from an initial level. Individual Differences in Slopes adds random intercepts and slopes across persons, including correlated initial level and rate of change. State-Dependent Change (Discrete Time) extends this to generate data for state-dependent change in discrete time, so trajectories reach a plateau rather than rise forever.

Section: Univariate Dynamic Systems in Continuous Time moves to continuous-time modelling in `ctsem`, introducing differential equations, simulation, state-space models, and the software workflow, using the basic state-dependent model as a starting point. State-Dependent Change with Fluctuations introduces system noise, distinguishing process change from measurement error.

With some of the modelling basics addressed, Section: Model Evaluation and Comparison demonstrates tools for checking model adequacy – a key step to ensure we can interpret model outputs sensibly. This section demonstrates a process of iterative model improvement starting from a misspecified model using a range of visual checks such as prediction plots, explained variance, and different forms of posterior-predictive check.

Section: Modelling Multivariate Systems returns to modelling and introduces coupled processes, where each partner in a dyad influences the other. The same cross-effect structure could apply to concepts such as linked symptoms, cognition and physiology, parent-child dynamics, or interacting behaviors.

Section: Individual Differences in System Dynamics shows how random-effects can combine with stable between-person variables such as time together to create heterogeneous system dynamics such as varied cross-effects between partners — this approach is important whenever dynamics or measurement parameters are expected to differ between individuals, due to either known or unknown stable factors.

Section: Interventions and External Events introduces approaches to model interventions and external shocks and estimate their influence on dynamic processes.

Section: Time-Varying Parameters considers two further extensions in which dynamics themselves vary over time: Latent State Moderation of System Parameters shows how a latent state's current level can moderate dynamic parameters, and Persistent Intervention routes external inputs through a separate latent accumulation process, leading to persistent shifts in system dynamics due to an intervention.

Readers less familiar with dynamic systems modelling may wish to focus on Section: Modelling Univariate Growth, Section: Univariate Dynamic Systems in Continuous Time, and Section: Model Evaluation and Comparison, leaving other sections for a later pass. More experienced readers may instead prefer to skip or skim Section: Modelling Univariate Growth, but consider that Section: Time-Varying Parameters contains relatively advanced material.

Modelling Univariate Growth

We start by considering a hypothetical patient named Alex. Alex started going to couples therapy and has agreed to track their affect once every seven days (at equidistant time intervals) for 21 weeks, through a mobile app. We aim to conceptualize ways that Alex's affect develops during this time. To begin with a minimally dynamic concept, we suppose that Alex's affect increases at a steady rate throughout the course of therapy.

Linear Change

To turn our conceptual idea of change into a statistical model we can fit to data, we need an appropriate statistical model. For a steady increase in affect, a linear regression model makes sense – this models how Alex's observed affect (y) changes at a steady rate (B) as a function of time (t) from an initial affect state (η_{t0}), with some random deviations from this process:

$$y(t) = \eta_{t0} + Bt + \epsilon(t)$$

At any time t, Alex's observed affect equals their starting level (η_{t0}) plus a constant amount of growth per unit time (Bt), with some random error ($\epsilon(t)$). The parameter B represents how much affect is expected to increase each week. In this first example, we treat deviations $\epsilon(t)$ as random noise, unrelated to earlier or later time points — this could be considered as measurement error or some form of variation that does not affect the overall trajectory of the process; later sections distinguish such unsystematic error from system noise which changes the trajectory itself.

Simulation

We can obtain a better understanding of the growth process implied by our model (and whether it matches our conceptual idea) by generating data from the model (along with specific parameter values) and visualizing the resulting trajectory of affect.

The first code chunk below specifies a plotting function that we will re-use throughout the tutorial — details on this are beyond the scope of this tutorial, but it contains a set of instructions for how to plot the data using the `ggplot2` package (Wickham, 2011) — run this code before proceeding with further code chunks.

```
plot_trajectory <- function(
    data, x="Time", y="Affect", x_label="Weeks", y_label="Affect",
    color_var=NULL, group_var=NULL, show_legend=FALSE, y2=NULL) {
  p <- ggplot(data, aes_string(x=x))
  if (!is.null(group_var)) p <- p + aes_string(group=group_var)
  if (is.null(y2)) {
    p <- p + aes_string(y=y) + geom_point() + geom_line()
  } else {
    p <- p + geom_line(aes_string(y=y)) +
      geom_line(aes_string(y=y2), linetype="dashed") +
      geom_point(aes_string(y=y))}
  if (!is.null(color_var)) {
    p <- p + aes_string(color=color_var)
```

```
    if(color_var %in% names(data) && is.numeric(data[[color_var]]))
      p <- p + scale_color_gradient(low="red", high="blue")}
  p <- p + theme_bw() + labs(x=x_label, y=y_label)
  if(!is.null(color_var) && is.null(y2)) p<-p+labs(color=color_var)
  if(!show_legend) p <- p + theme(legend.position="none")
  p}
```

Moving to the simulation code, we first specify the number of time points to generate data for (`Time <- 0:20`). Second, we set the initial state of affect at the beginning of the growth process, i.e.,the beginning of therapy (`t0Affect <- 5`). Third, we specify the linear regression model for a single person, whose affect starts at an initial level (`t0Affect`) of 5 and increases steadily by 0.51 during every weekly observation (`Time*.51`). We use an arbitrary affect scale where larger values indicate more positive affect; the starting level, slope, and noise values are chosen to make the implied trajectory visible rather than to represent calibrated empirical values. Fourth, we add random noise by sampling values from a normal distribution with a mean of 0 and a standard deviation of 0.2 (`rnorm(n=length(Time), mean = 0, sd = .2)`). These noise values are added to each generated affect value. Fifth, we create a data frame to store the number of time points (i.e.,weeks) and the generated affect values for each time point. Finally, we use the created function `plot_trajectory` to plot the data, illustrating one sample of the model-implied affect process over the course of 21 weeks of therapy.

```
Time <- 0:20 # Generate time points for each measurement occasion
t0Affect <- 5 # Initial level of mood at week 0

# Specify linear model where affect increases by 0.51 each week
Affect <- t0Affect + Time*.51 +
  rnorm(n=length(Time), mean = 0, sd = .2)

# Create a data frame to store the generated data
data <- data.frame(Time = Time, Affect = Affect)

plot_trajectory(data)
```

Figure 1

Generated Linear Affect Trajectory for a Single Person.

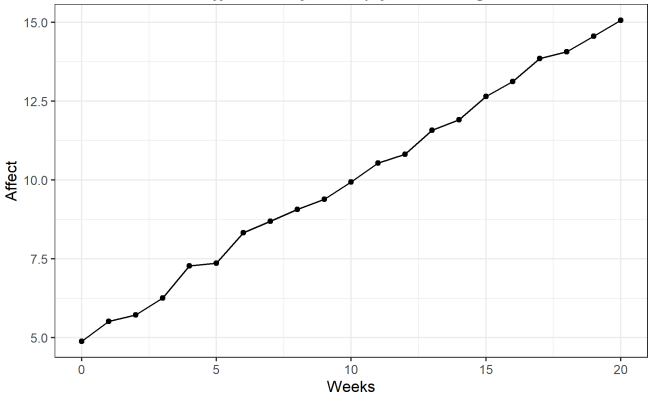


Figure 1 shows that our linear model provides a good representation of the idea that Alex's affect increases at a steady pace over time, with some random deviations from this.

Individual Differences in Slopes

By viewing Alex's model-implied growth process it strikes us that we have no idea whether their rate of change is slow or fast relative to other people who go to couples therapy. That is, we may want to know if couples therapy leads to a different amount of change for different people, and if differences in people's rate of change are related to their initial level of affect. For instance, people who have a much lower initial level of affect than Alex may benefit more from couples therapy. To understand where Alex sits relative to other patients, we need to introduce individual differences into our model.

The equation for a linear model which allows people to vary in their initial affect level and rate of change looks very similar to the linear regression model presented above but includes an i subscript, this indicates that each person has their own initial level (η_{t0i}) and rate of change ($B_i t$) of affect:

$$y_i(t) = \eta_{t0i} + B_i t + \epsilon_i(t)$$

The subscript i indicates that each person i has their own parameters. Person i's affect starts at their own initial level (η_{t0i}) and grows at their own rate (B_i). For example, Person 1 might start at affect level 3 and grow by 0.7 units per week, while Person 2 might start at affect level 6 and grow by 0.4 units per week. This allows us to model both within-person change and between-person differences.

This model is commonly termed a linear mixed-effects model (Bates et al., 2015), because it can include both fixed effects (parameters that are assumed to be the same across all subjects, often interpreted as population-level effects) and random effects (parameters that allow for individual variations, such as subject-specific intercepts η_{t0i} or slopes $B_i t$).

Simulation

We now generate data from our linear mixed-effects model. The code below is used to generate data for multiple subjects (`NSubjects`) whose affect is measured 21 times, once per week for 21 weeks (`times <- seq(from=0, to=20, by=1)`). To generate a different initial state for each subject, we sample initial states from a normal distribution of initial affect levels with a mean of 5 and a standard deviation of 2 (`t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)`). The values here are arbitrary and can be tweaked to adjust the standard deviation of initial affect (`sd`), and the population average initial affect level (`mean`). To match our hypothetical scenario, where people's rate of growth in affect is related to their initial level, we construct each person's rate of change as a weighted combination of two standardized pieces: their initial affect level and an additional random slope component. The weight on initial affect is the desired intercept-slope correlation (`initSlopeCor`), and the weight

on the random component keeps the total slope variance at 1 in expectation. We then shift and scale this standardized slope using `slopeMean` and `slopeSD` to obtain the final slope for each subject, correlated with initial affect.

```
# Generate data for multiple subjects with individual differences
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #measurement occasions
Nobs <- length(times) #number of observations per subject
initSlopeCor <- -.8
#sample initial affect for each subject from normal distribution
t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)
#sample slopes for each subject
slopeMean <- .5
slopeSD <- .2
zInitialAffect <- scale(t0Affect) #standardized initial affect
randomSlopePart <- rnorm(n = NSubjects) #extra individual differences
randomSlopePart <- scale(randomSlopePart) #standardized slopes
zTimeEffect <- #standardized slope correlated with initial affect
  initSlopeCor * zInitialAffect + #correlated part
  sqrt(1 - initSlopeCor^2) * randomSlopePart #uncorrelated part
timeEffect <- slopeMean + slopeSD * zTimeEffect
```

We can now create an empty data frame that can later hold the data we generate for each subject and each observation. To fill the data frame, we create two nested loops. The first loop (`for(subi in 1:NSubjects)`) allows us to go through each subject (`subi`) one by one. For each subject selected by the first loop, we initialize the second loop (`for(obsi in 1:Nobs)`) that allows us to go through each observation (`obsi`). Within this loop, we fill in each row in our data frame, which corresponds to an observation. For each observation, we generate information about a person's affect level, the current time point (week), and their subject identifier. To keep track of the current observation for each iteration of our loop, we initialize a row count at 0 (`row <- 0`) and then add 1 to this running row count (`row <- row + 1`). To reflect the imperfect measurement of affect, we add some random noise to the affect data that we generated, by sampling random values from a normal distribution with a mean of 0 and a standard deviation of 1 (`data$Affect <- data$Affect + rnorm(n=nrow(data), mean = 0, sd = .2)`).

```
#create empty data.frame to fill step by step
data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect = rep(NA,NSubjects*Nobs))

row <- 0 #initialize row counter to track current row
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1 #add 1 to row to reflect current row
    # current affect equals initial affect and effect of time:
    data$Affect[row] <- t0Affect[subi] +
      times[obsi] * timeEffect[subi]
    data$Time[row] <- times[obsi] #store time point for this row
    data$Subject[row] <- subi #store subject id for this row
  }
}
data$Affect <- data$Affect +
  rnorm(n=nrow(data), mean = 0, sd = .2) #measurement error
data$InitialAffect <- rep(t0Affect, each = Nobs)

plot_trajectory(data, color_var = "InitialAffect",
  group_var = "Subject", show_legend = TRUE)
```

Figure 2

Affect Trajectories for 20 Subjects with Individual Differences in Initial Level and Rate of Change.

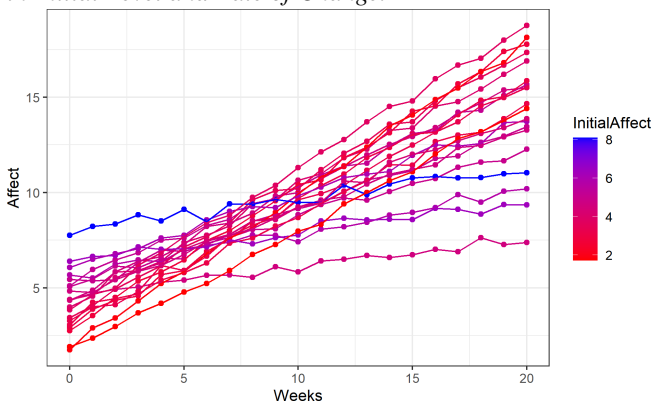


Figure 2 shows that the correlated initial state and rate of change capture the idea that individuals who start with lower affect levels may improve faster over time.

Model Fitting

We can now fit our model to the data we generated and interpret how well it captures the patterns in our observations. To specify and fit our linear mixed-effects models, we will use the lme4 package in R (explicitly including intercepts, although they would be included by default):

```
library(lme4)
lme_model <- lmer(Affect ~ # predict Affect
    1 + # a fixed effect for the intercept
    Time + # with a fixed effect of time
    #random intercept and random slope of time per subject:
    (1 + Time | Subject),
  data = data) # specify data to fit model to
```

After fitting the model to our data, we can summarize the parameter estimates via `summary(lme_model)`. For this simple affect process, we can get a grasp on the model-implied trajectory through the numerical estimates offered by lme4's output table. We can find the population-level estimates for the intercept and slope values under "Fixed effects". The variance of the subject-specific deviations from the population average can be found under "Random effects". The correlation between the individual differences in the intercept and slope values is also under the "Random Effects" section (do not be misled by the "Correlation of Fixed Effects" heading, it does not reflect individual difference correlations). For more information about lme4 and its functionalities we refer the reader to the package documentation (Bates et al., 2015) https://cran.r-project.org/web/packages/lme4/lme4.pdf.

Visualizing Predictions from lme4

Visualization helps build intuition about the process our models imply and allows us to detect sources of model misfit

that may be difficult to identify if solely relying on numerical fit indices (see Gabry et al. (2019) for examples). For our lme4 model, we can generate two plots showing: (1) the model's predictions (solid line) and its associated uncertainty (shaded ribbon) based only on the estimated model parameters, thereby representing all possible subjects; (2) predictions based on the model parameters and all data of a single subject (subject 3). The code below uses the `bootMer` function to generate bootstrap (i.e., shuffled resamples) predictions from the model, and then uses the `ggplot2` package to visualize predictions and uncertainty.

```
# Use bootMer to generate bootstrap (i.e., many) predictions
boot_preds <- bootMer(lme_model, FUN = predict, nsim = 100)

# Combine original data, predictions, and confidence intervals:
newdata <- data.frame(data,
  pred = apply(boot_preds$t, 2, mean),
  lower = apply(boot_preds$t, 2, quantile, 0.025),
  upper = apply(boot_preds$t, 2, quantile, 0.975))

# Use ggplot2 to visualize the predictions and uncertainty
ggplot(newdata[newdata$Subject==3,], aes(x = Time)) +
  geom_line(aes(y = pred, color = "Predicted")) +
  geom_ribbon(aes(ymin = lower, ymax = upper,
    fill = "95% Confidence"), alpha = 0.2) +
  geom_point(aes(y = Affect, color = "Observed"), size = 2)+
  theme_bw()+ylab('Affect')
```

Figure 3

LME4 Model Predictions and Confidence Interval Based on Parameter Estimates Only, Subject 3.

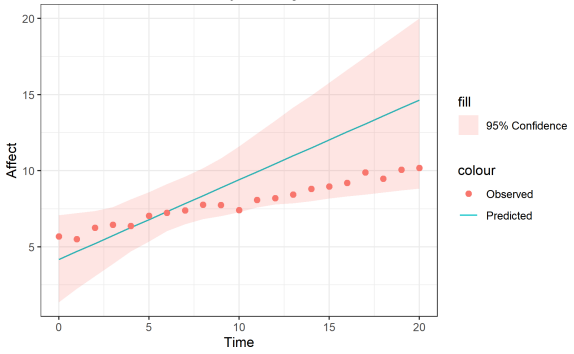


Figure 3 shows how well information solely from the estimated model parameters can reproduce our pattern of observations. The red dots show the observed affect values at each time point, and the solid line shows the model implied expectation of affect, with the shaded area surrounding the line showing the variation / uncertainty about this trajectory. We can see that by solely relying on the model expectation, our model does a relatively poor job of predicting any specific subjects' actual data points. This is related to partial pooling: before conditioning on the subject's own observations, the model relies on the population distribution of trajectories rather than on the specific intercept and slope for this person.

We can also examine model predictions conditional on both parameter estimates *and* a subjects' entire data:

```
preds <- predict(lme_model,  se.fit = TRUE) #get predictions

#combine original data and predictions with confidence intervals
newdata <- data.frame(data, preds,
  lower = preds$fit - 1.96 * preds$se.fit,
  upper = preds$fit + 1.96 * preds$se.fit)

# Use ggplot2 to visualize the predictions and uncertainty
ggplot(newdata[newdata$Subject==3,], #just for subject 3
  aes(x = Time, y = fit)) +
  geom_line(aes(color = "Predicted")) +
  geom_ribbon(aes(ymin = lower, ymax = upper,
    fill = "95% Confidence"), alpha = 0.2) +
  geom_point(aes(y = Affect, color = "Observed"), size = 2)+
  theme_bw()+ylab('Affect')
```

Figure 4

LME4 Model Predictions and 95% Confidence Interval for Subject 3 Conditional on All Observed Data.

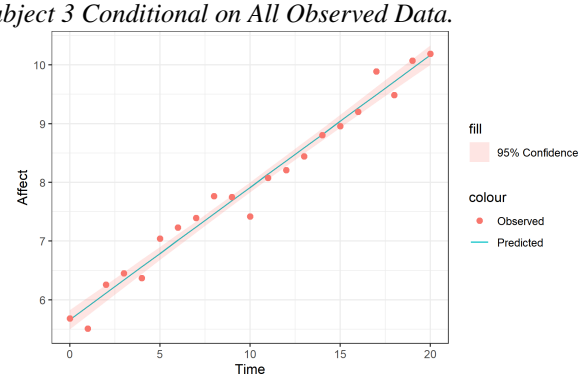


Figure 4 shows the model's predictions after it has learned from all the observations in our dataset. So, data from past, present, and future observations are used by the model to predict each observation. Unsurprisingly, the predictions are much better — remaining differences are due to the unsystematic noise term.

State-Dependent Change (Discrete Time)

By formalizing our linear growth process for Alex and other patients, we can inspect what our assumptions imply. In the simple simulated example above, the negative association between initial affect and rate of change implies that people with a lower initial level of affect can have a higher affect level at the end of the 21 weeks of therapy compared to people who started with higher affect. This means that if Alex entered therapy with more severe affect problems they would come out of therapy with a higher level of affect than if they entered with less severe affect problems. Moreover, if we take our model with a steady rate of growth seriously, then this would imply that Alex could endlessly increase their affect levels by continuing therapy. Both of these predictions seem unrealistic. So we have to rethink our model of affect (or interpret results from it **very cautiously**).

A more defensible prediction could be to expect that people's rate of growth slows down as their affect improves. This

would mean that people with a low initial level of affect would increase faster at the start, relative to people with a higher initial level of affect, but their growth rate would slow down as their affect improves. This is a conceptual shift from a deterministic trend indexed only by time to a state-dependent process: change depends on the current level of affect, or change is 'state-depdendent'. It also has substantive implications. If a person's affect starts above the implied long-run level, the same model predicts movement downward rather than continued improvement (a more realistic scenario potentially relates individual differences in the start point to long-run level). In a statistical model, we can implement such a (nonlinear) growth process by allowing each person's slope to vary not just as a function of where they start, but also as a function of where they are *at every point in time*.

We can model such 'state-dependent' growth by computing the observed level of affect at each time point ($\eta(t)$) as a function of affect at the previous time point ($A\eta_{t-1}$), plus a constant value (B) that is added to each observation:

$$\eta(t) = A\eta_{t-1} + B$$

At each time point t, affect depends on two components: (1) a fraction A of the previous affect level, and (2) a constant input B. The parameter A controls state dependence — if A is positive but less than 1 (e.g., $A = 0.8$), affect carries forward from the previous time point but at a reduced rate, creating a dampening effect — with only this state-dependency fraction, no matter the initial value of affect (whether positive or negative), the process would always converge towards zero. Including the constant input B allows the process to converge towards a different, non-zero value (which is not exactly B, but a function of both A and B). In case of an A between 0 and 1, and a B value greater than zero, the process will converge towards a positive value — whether the process starts above this value or below.

With this equation we can emulate a process where the rate of growth in affect dampens throughout the course of therapy. Our code will allow us to visualize the model-implied trajectory that this equation implies, but we can also build some intuition by solving this equation for a small number of time points.

Consider 3 time points T0, T1, and T2. Assume an autoregressive coefficient (A) at 0.8, which means that 0.8 of the previous observation will be added to the current observation. Let's also add a constant of 2, as our continuous intercept (B) which is added to every observation.

We initialise the process at T0 by setting initial affect to 5 ($\eta(0) = 5$). At T1, $0.8 * 5 + 2$ results in affect of $\eta(1) = 6$. Moving to the next time point, T2, $0.8*6+2$ gives $\eta(2) = 6.8$. So from T0 to T1 our affect level grew by 1 unit, but from T1 to T2 it only grew by 0.8. The rate of growth is dampening as consequence of a change in the prior level of affect.

Simulation

Below we provide code that does this computation for all of our 21 weeks. First we set the degree to which the current level of affect is dependent on the prior level of affect (i.e.,state dependence, `A <- .8`). Then we set the constant value which is added to each observation (`B <- 2`). We then set an initial affect level to initialize our process (`t0Affect <- 5`). An if-else statement is used so that the first observation sets the initial level of affect (`if(i==1) Affect[i] <- t0Affect`) and each subsequent observation has an affect value generated by our model equation (`else Affect[i] <- A*Affect[i-1] + B`).

```
times <- seq(from=0, to=20, by=1) #time points of measurement
A <- .8 #autoregressive coefficient / state dependence
t0Affect <- 5
B <- 2 #continuous intercept
Affect <- rep(NA,length(times)) #create empty affect vector
for(i in 1:length(times)){ #for each measurement occasion
  #if first time point, set to initial affect:
  if(i==1) Affect[i] <- t0Affect
  #otherwise, use autoregression and continuous intercept:
  else Affect[i] <- A*Affect[i-1] + B
}

plot_trajectory(data.frame(Affect=Affect, Time=times))
```

Figure 5

Trajectory from Discrete-Time Autoregressive Model.

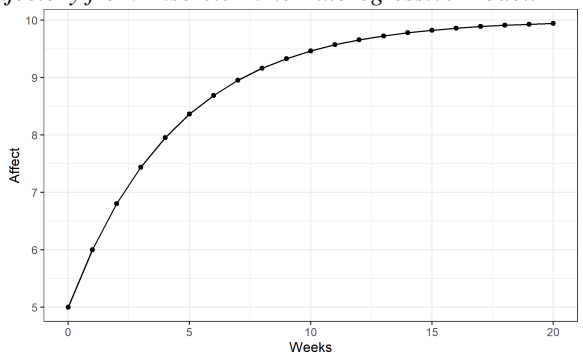


Figure 5 shows a the nonlinear increase in affect over time. This is a better representation of a reality where therapy cannot make you infinitely happy, but may help recover from a low point.

Univariate Dynamic Systems in Continuous Time

Our model from State-Dependent Change (Discrete Time) provides a more realistic depiction of Alex's growth in affect from week to week. But what happens in between each of our weekly observations? One possibility is that affect stays flat and suddenly jumps during each of our weekly observations. That is, change only happens when we are looking. If we try to imagine such a discretely changing affect process, it would look like a staircase of affect. The alternative possibility is

that people's affect changes continuously across time, forming a coherent trajectory. The rate of change that we observe from week to week, is then an aggregate of all the moment-to-moment changes that happen between our weekly observations. If the second option is more akin to our expectation of reality, then a continuous-time framework can better represent the process and provide parameters more closely aligned to theoretical concerns. This becomes particularly important when modelling multiple interacting processes or when dealing with data collected at varying time intervals (For expanded details on such issues we refer the reader to Driver, 2025; Aalen et al., 2016; Kuiper & Ryan, 2018; Ryan & Hamaker, 2022; Voelkle et al., 2012).

Differential Equations

Differential equations are a natural way to represent processes that evolve continuously. A researcher does not need a deep technical understanding of differential equations to build useful intuition: the key shift is from modelling a variables value at the next observation based on the current observation, to modelling how fast the variable is changing at the present moment based on its current value. One way to soften this shift is to start with average change over a finite interval. As the interval Δt becomes smaller, this average rate approaches the instantaneous rate of change, written as the derivative $\frac{d\eta}{dt}$. The differential equation then describes what determines that momentary rate of change.

We can model an affect process that changes at a steady rate in continuous time, akin to our first example where we used a linear regression model — but this time using a differential equation. The major difference here is that our outcome variable (left-hand side of equation) is the *rate of change*, rather than the *level*, of affect — level was the outcome in our discrete time (regression) representation. This rate of change in affect (η) at a given moment in time t is denoted by the derivative $\frac{d\eta}{dt}$. We can think of the rate of change at a given moment in time as what we would get if we were to glance at a speedometer in our car. The velocity that we would see, would tell us how fast our current position is changing at a particular moment in time. Since our model is linear, the rate of change at a given moment is given by a constant value B — there is no variation in the rate of change (or slope), it remains constant over time. Thus, the differential equation for a linear model is:

$$\frac{d\eta}{dt} = B$$

The notation $\frac{d\eta}{dt}$ reads as "the rate of change of η with respect to time t" — think of it as affect's "velocity" at any instant. If $B = 0.51$, then at every moment, affect is increasing at a constant rate of 0.51 units per week. This is like a car traveling at a constant speed, always travelling 100 km/h. Similarly, no matter Alex's current affect level, it's always increasing by 0.51 units per week.

For readers who remember basic calculus, this is the derivative of a linear function: if $\eta(t) = \eta(0) + Bt$, then the derivative with respect to time is B. The continuous-time equation therefore describes the same linear idea, but in terms of the instantaneous slope rather than the level at a particular observation.

If we want to pick up where we left off and represent our state-dependent growth model in continuous time, we can add a term that allows the rate of growth at a given time point to depend on the current state of affect (i.e.,a state dependency):

$$\frac{d\eta}{dt} = A\eta(t) + B$$

Now the rate of change (velocity) depends on where affect currently is, as well as on the constant input B. The term $A\eta(t)$ means the rate of change is proportional to current affect level. If A is negative (e.g., $A = -0.2$), higher affect leads to slower growth (or even decline), creating a self-regulating system — just as we saw in our discrete time model when the autoregression coefficient A was positive but less than 1. Here in the continuous-time case A is still a regression coefficient, but the outcome variable is now the *rate of change in affect*, rather than the *level of affect at a new time point*. The term B is still a constant upward push.

Simulating Continuous-Time Dynamics

To generate data for this continuous growth process in R, we can approximate the solution to the differential equation using a 'Euler-Maruyama' method (Higham, 2001) (or technically in this case without system noise, we would call this just the 'Euler method'). This method works by breaking continuous time into small time slices (steps) and updating the state of our growth process at each step based on the model's dynamics.

The differential equation defines an instantaneous rate of change, but simulation requires us to approximate the trajectory over finite steps. To generate data, we first define values for our model parameters. This includes parameters very similar to the discrete-time version: (1) the degree to which the current level of affect influences its own rate of change (`A`, continuous time state-dependence). (2) A constant input to the growth process (`B`, continuous intercept), and an initial level of affect (`t0Affect`). Then we create an empty vector to store the affect values given by our for-loop which implements the 'Euler-Maruyama' method (`Affect <- rep(NA,Nobs)`). The for-loop sets the initial level of affect for the first observation (`i=1`) to initialize the process (`if(i==1) Affect[i] <- t0Affect`). For each subsequent observation (`i > 1`), we compute the current level of affect by summing the *rate of change* at the prior time point (`dAffect = A*Affect[i-1] + B`) with level of affect at

the prior time point (`Affect[i-1]`). That is, `Affect[i] <- Affect[i-1] + dAffect * 1`. To add the size of the desired time step into the equation, we multiply the rate of change by the size of the time step, in this case `dAffect` is multiplied by 1 (a pedagogically helpful but crude choice), meaning that the rate of change is equivalent to the entire rate of change between each observation.

```
times <- seq(from=0, to=20, by=1) #measurement times
Nobs <- length(times) # number of observations per subject
A <- -.2 # continuous time state dependence
tOAffect <- 3 # set initial level of affect
B <- 2 # continuous intercept
Affect <- rep(NA,Nobs) # create empty affect vector

for(i in 1:Nobs){ # for each time point
  if(i==1) Affect[i] <- tOAffect #if first time, initial affect
  else{ # otherwise compute slope of affect at earlier time point
    dAffect <- A*Affect[i-1] + B
    # then update affect using slope and time step
    Affect[i] <- Affect[i-1] + dAffect * 1
  }
}
plot_trajectory(data.frame(Affect=Affect, Time=times))
```

This is a crude approximation however — it assumes the rate of change is constant over each time interval of 1, when in fact our model specifies that the rate of change varies whenever affect changes, which is continuously. We use this rough approximation first for intuition, then refine it. To better approximate a continuous time solution, we can take smaller steps in time. But more steps means that the necessary computation will be more intensive. So let's start with a small number of intermediary steps, say 10 steps in the time between our observations, to better approximate the continuous time process `Nsteps <- 10`. Now, we introduce another loop, within our previous for-loop, which splits the interval between each observation (e.g. week) into 10 smaller steps (`istep`). The first line within this loop represents the differential equation which gives us the rate of change at the prior time point (`dAffect = A*Affect + B`). The line below it, updates the current level of affect based on the estimated rate of change at the prior time point multiplied by 1/10 (i.e.,`1/Nsteps`). The effect of this multiplication is to give us the rate of change over the smaller step of time we chose. In essence, this scales the rate of change from the full unit of time (e.g., week), given by the preceding equation, to one-tenth, allowing us to better approximate the underlying continuous process.

```
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
A <- -.2 #continuous time state dependence
tOAffect <- 3
B <- 2 #continuous intercept
Affect <- rep(NA,Nobs) #create empty affect vector
Nsteps <- 10 #number of steps between each observation

for(i in 1:Nobs){ #for each time point
  if(i==1) Affect[i] <- tOAffect #init with initial affect
  else{ #compute new affect state by sequence of small steps
    #initialise with state at previous time point
    currentAffect <- Affect[i-1]
    for(stepi in 1:Nsteps){ #take Nsteps for each observation
      #compute slope of affect at earlier time point
      dAffect <- A*currentAffect + B
      #update state using slope and time step
      currentAffect <- currentAffect + dAffect * 1/Nsteps
    }
    Affect[i] <- currentAffect
  }
}
```

State-Dependent Change with Fluctuations

By using our state-dependent model to predict Alex's affect trajectory, we would assume that their affect would change in a smooth deterministic manner. However, real-world processes are typically subject to unmodelled influences that lead processes to fluctuate in meaningful and persistent ways — perhaps Alex wakes up with a headache one day, or has a particularly nice conversation with a friend. These influences may not alter their long term trajectory, but can induce an effect that persists for at least some length of time. Such fluctuations are distinct from variation we might attribute to *measurement error* where we typically assume that imperfections in our measurements are random and independent at each occasion of measurement.

We can model random fluctuations in the affect process by adding a 'system noise' term (sometimes called process noise or latent variance) to our differential equation. In our model so far, affect depends only on its current level and a fixed intercept, but in reality many other variables also matter: sleep, minor hassles, chance encounters, and countless other influences we do not measure or include explicitly. System noise stands in for the combined effect of these unmodeled variables: it allows the true process state to be pushed up or down by influences outside the equation. Unlike measurement error, which affects only the observation, system noise changes the underlying affect level itself, and those shifts persist because the dynamics carry them forward over time. With our current equation, the rate of change in affect $\frac{d\eta}{dt}$ is purely deterministic, meaning that how fast affect changes at a given moment is entirely determined by the terms in the right-hand side of the differential equation ($A\eta(t) + B$), with no allowance for such unmodeled influences.

To include system noise, we add a term that introduces some randomness to the rate of change at each moment. Meaning that aside from the deterministic effects there will be some random ups and downs in Alex's growth rate. To add this randomness, we can introduce a noise term that is sampled from a normal distribution. This turns our (ordinary) differential equation into a stochastic differential equation (Kloeden & Platen, 1992).

Our continuous time process evolves over arbitrarily small steps in time. At each step in time, a portion of randomness is injected into the rate of change. The magnitude of this randomness is scaled by the system noise effect G, defining the volatility of the process. Thus, our rate of change in affect $\frac{d\eta(t)}{dt}$ is a combination of some deterministic effect

$(A\eta(t) + B)$ and some random effect $(GdW(t))$, over each interval dt. Because of the stochastic nature of the random noise term, it is not possible to analytically define its rate of change at any point in time. Thus, stochastic differential equations are often written a bit differently to regular differential equations, with the dt term moved to the right-hand side of the equation, and only applied to the deterministic portion:

$$d\eta = (A\eta(t) + B)dt + GdW(t)$$

Simulation

We can add system noise to our continuous time model by updating the data generating code using the Euler method discussed in the previous section. The amount of system noise occuring over each time step will be added to the affect level at each step. Thus, when we update a person's current affect state at each small time step we add some noise. This noise is sampled from a normal distribution with a mean of 0 and a standard deviation of 1 divided by `Nsteps` (this time 100 steps). Before being added to the current affect state, each draw of random noise is multiplied by `G` (the system noise coefficient). Thus, the higher the `G` value the more noisy the process becomes.

To generate data for multiple individuals we have to make further adjustments to the earlier code. For each participant we sample their initial affect level from a normal distribution with mean 5 and standard deviation of 2 (`t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)`). Then we create an empty data frame to fill step by step. Finally, we add a loop to repeat the computation of the affect process for each subject (`for(subi in 1:NSubjects)`).

```
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)
A <- -.1 #continuous time state dependence
B <- 1 #continuous intercept
G <- 0.2 #system noise coefficient

#create empty data.frame to fill step by step
data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect = rep(NA,NSubjects*Nobs))

Nsteps <- 100 #steps between each observation
row <- 0 #init current row tracker
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1
    if(obsi==1) Affect <- t0Affect[subi] #init with initial affect
    if(obsi>1){ #else compute new state via small steps in time
      for(stepi in 1:Nsteps){ #every step, do...
        dAffect <- A*Affect + B #compute affect slope
        Affect <- Affect + #new state = old state +
        dAffect * 1/Nsteps + #deterministic change +
          G * rnorm(n=1, mean=0, sd=sqrt(1/Nsteps)) #random change
      }
    }
    data$Affect[row] <- Affect #input affect data
    data$Time[row] <- times[obsi] #input time data
    data$Subject[row] <- subi #input subject data
  }
}
#add measurement error
data$Affect <- data$Affect+rnorm(n=nrow(data), mean = 0, sd = .2)
#plot trajectories
plot_trajectory(data, color_var = "as.factor(Subject)")
```

Figure 6

Affect Trajectories with System Noise and Individual Differences.

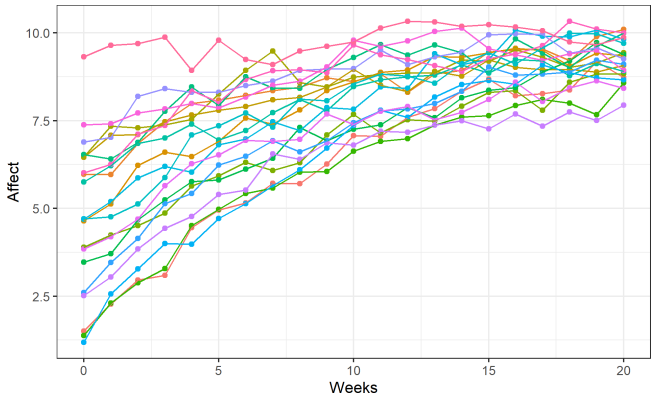


Figure 6 gives a more realistic depiction of affect dynamics, with fluctuations around the smooth, state-dependent increase in affect over time. This model captures the idea that affect processes are subject to both predictable trends and random fluctuations, reflecting some of the complexity of real-world dynamics.

State-Space Models

To fit a continuous-time model with state-dependence and system noise to data we now shift from generative simulation to estimation in a 'state-space' framework (Durbin & Koopman, 2012; Harvey, 1989; Kalman, 1960). This shift adds a layer of abstraction. Instead of treating the observed affect score as the process itself, we distinguish a latent dynamic process from the observations that measure it. The latent process evolves continuously according to the dynamic model, while the measurement model links that latent process to the observed affect scores at the times they were measured.

The examples in this tutorial use simple single-indicator measurement models for pedagogical clarity. Thus, the tutorial emphasizes the dynamic state-space part of continuous-time structural equation modelling more than full multiple-indicator latent-variable measurement modelling. Multiple indicators, factor loadings, and more elaborate measurement structures can be specified in this framework, but they are not the focus of this tutorial — for a tutorial focused on R and SEM multiple-indicator measurement modelling see Rosseel and Vidal (2025).

ctsem: A Flexible Tool for Dynamic Systems Modelling

The R package `ctsem` (Driver et al., 2017; Driver & Voelkle, 2018a) provides a software framework for specifying and fitting continuous-time (CT) and discrete-time (DT) dynamic models. It enables researchers to model the behavior and evolution of processes over time, and its combination of features can offer some unique advantages for analyzing longitudinal data.

Software Installation

Before starting, ensure your system is ready to use `ctsem`. Follow these steps:

1. **Install R and RStudio**: Download and install R (https://cran.r-project.org/) and RStudio (https://posit.co/products/open-source/rstudio/) or alternative interfaces.
2. **Configure c++ for model compilation with Stan**: `ctsem` relies on Stan (Carpenter et al., 2017) for log likelihood calculations. Most of the models we will fit do not require compilation, but some of the more complex models we will fit do. Ensure you have a compatible C++ compiler installed:

- **Windows**: Install RTools (https://cran.r-project.org/bin/windows/Rtools/).
- **Mac**: Install Xcode command line tools by running `xcode-select --install` in the terminal.
- **Linux**: Install `g++` and other build tools using your package manager (e.g., `sudo apt install build-essential`).

3. **Install Required Packages**:

```
install.packages("ctsem", dependencies=TRUE)
```

Model Fitting in ctsem

To fit the data we generated in Section: State-Dependent Change with Fluctuations with `ctsem`, we need to specify our model using the `ctModel` function. The code below is annotated for convenience, for more information about the specification of a continuous-time model see the ctsem manual https://cran.r-project.org/web/packages/ctsem/ctsem.pdf (Driver et al., 2025). Here we will provide a quick description of some `ctsem` specifics — while argument names and specifics will vary the model structure generalizes to other software and approaches too.

Our `ctsem` model can be thought of as linking a latent (unobserved) 'true' process to observed (or 'manifest') variables, which are part of our data. Observed variables are treated as indicators of the underlying latent process, and are linked to the latent states via a measurement model. For the running example, this means that Alex's affect is assumed to evolve continuously between observations, while the observed ratings are (potentially) noisy snapshots of that underlying process.

In this case our latent dynamic model is given by the stochastic differential equation:

$$d\eta = (A\eta(t) + B)dt + GdW(t)$$

This model is quite general, and can be formulated as very large matrices (when e.g., there are many latent processes interacting with each other and or many observed variables). As such `ctsem` expects matrices as inputs for each of the different model components, but also contains some shortcuts to make it easier to specify the model. Model components can be either fixed to specific values (input a numeric value) or estimated from the data (input a string with the name of the parameter to estimate). When we move on to matrices later, each matrix can be a mixture of fixed and estimated values. Here is how we specify our current dynamic model using the `ctsem` syntax:

1. The `DRIFT` argument specifies the state dependence A, and contains a freely estimated parameter we call `stateDependence`, making the rate of change depend on the current level of affect.
2. The `CINT` argument specifies the continuous intercept (B). We call the continuous intercept parameter `B` and only estimate a fixed effect (one value for all subjects). To switch off random effects (on by default for initial values and intercept parameters) we need to add `FALSE` into the 3rd 'slot' of the parameter, done by adding vertical bars after the parameter name, i.e.,`B||FALSE`. We do not need to modify the 2nd slot, so it is left empty.
3. The `DIFFUSION` argument specifies the system noise effect (G), and contains a freely estimated parameter `systemNoise`, allowing the model to capture random fluctuations around the model-implied trajectory.
4. The initial value of our latent process is treated as a random variable with its own mean and variance parameters: $\eta_{t0} = \mu_{0i} + \epsilon_{0i}$, where $\epsilon_{0i} \sim \mathcal{N}(0, \sigma^2)$. The `TOMEANS` argument, is used to specify the mean parameter for the initial level of the process, which we call `t0Affect`. By adding `TRUE` behind `t0Affect||` we are allowing for random effects (between-subject variance) around the population estimate of the initial level of the process in our sample — this would have been enabled by default in any case.

The latent process model is linked to our observations using a (linear) measurement model:

$$\text{Affect}(t) = \Lambda\eta(t) + \tau + \epsilon(t)$$

Where $\text{Affect}(t)$ is the observed variable affect. Λ is a matrix of regression weights used to link the latent process to our

observations, η is the latent level of affect, τ reflects a constant that can be used to shift the relationship between observations and a latent variable, and ϵ represents measurement error, and is distributed as a multivariate normal distribution with mean 0 and (usually diagonal) covariance matrix Θ. We specify the observation model in ctsem as:

1. The `LAMBDA` argument specifies the Λ matrix. In our case it is a 1x1 matrix with the value 1, meaning that the latent Affect (η) is directly and equally reflected in the observed Affect without any scaling.
2. The `MANIFESTMEANS` argument specifies the measurement intercept (τ), which we have set to zero, implying that there is no systematic offset in the measurement process.
3. The `MANIFESTVAR` argument specifies the standard deviations of measurement error (ϵ), which we call `residualSD`.

```
m_basic <- ctModel( #define the ctsem model
  # Specify features of the data / model type
  manifestNames = "Affect", #names of observed variables in data
  latentNames = "Affect", #names for latent processes
  time = 'Time', #name of time column in dataset
  id = 'Subject', #name of subject column in dataset
  type='ct', #use continuous-time (dt for discrete-time)
  # Specify model components
  MANIFESTVAR = 'residualSD', #SD of measurement error
  LAMBDA = matrix(1,nrow=1,ncol=1), #factor loading matrix
  MANIFESTMEANS=0, #set measurement intercept / offset to 0
  CINT='B||FALSE', #continuous intercept with *no* random effects
  TOMEANS='t0Affect||TRUE', #initial affect + random effects
  DRIFT = 'stateDependence', #state dependence
  DIFFUSION = 'systemNoise') #system noise
```

We can now fit the model and ask for a summary of the parameter estimates.

```
f_basic <- ctFit(datalong = data, model = m_basic)
```

```
summary(f_basic, parmatrices= FALSE) #some output disabled
```

```
Standardised residual covariance
---------------------------------
       Affect
Affect  0.096

Random-effects standard deviations
---------------------------------
          mean    sd  2.5%   50% 97.5%
t0Affect 2.181 0.344 1.615 2.156 2.931

Fixed-effects / Population means
---------------------------------
                   mean    sd   2.5%    50%  97.5%
t0Affect          4.755 0.498  3.787  4.758  5.730
stateDependence -0.105 0.007 -0.118 -0.105 -0.092
systemNoise       0.210 0.019  0.175  0.210  0.250
residualSD        0.195 0.015  0.167  0.195  0.226
B                 1.039 0.054  0.933  1.041  1.135

Note: covariance pars in sd / unconstrained cor form, see System
      Matrices (or ctSummaryMatrices function) for cor/cov.
-------------------------------------------------------------------------
Log posterior: -144.457
Log likelihood: -144.457
Number of parameters: 6
AIC: 300.914
```

The summary output provides several key sections for interpreting model fit and parameters:

- Standardised residual covariance: Unexplained (co)variation after the model's predictions have been accounted for.
- Random-effects correlations: Correlations between random effects; only shown when multiple random effects are estimated.
- System Matrices: Expanded matrix summaries when `parmatrices = TRUE` (suppressed here with `parmatrices = FALSE`).
- Random-effects standard deviations: Population standard deviations for random effects.
- Fixed-effects / Population means: Estimates for the system and measurement model fixed-effect parameters.

From our summary table, we can see that the estimated coefficients are not exactly the same as the true values, due to sample variation in the data. The model does however capture the general trend of affect dynamics over time, with parameter estimates close to our specified generating model.

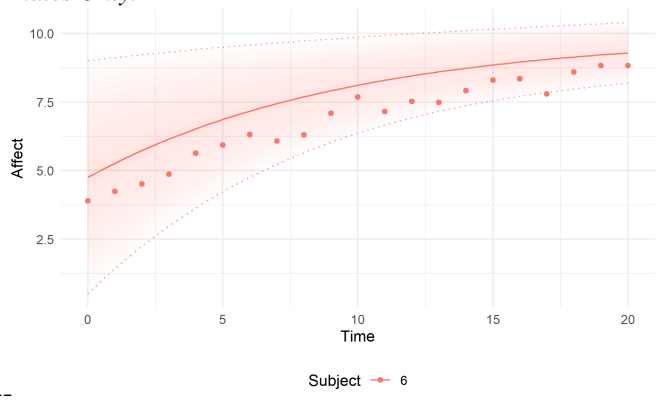
Visualizing Predictions with ctsem

Using `ctsem` we can easily plot the model predictions and associated uncertainty. Our predictions can be conditional on all, none, or some of the individual subjects' observed data. First let's look at the predictions based solely on the parameter estimates, for an arbitrary subject, number 6 (multiple subjects could be specified in the `subjects` argument).

```
ctPredict(fit= f_basic, plot = TRUE, subjects = 6, removeObs = TRUE)
```

Figure 7

ctsem Predictions for Subject 6 Based on Parameter Estimates Only.



Just like in the case of our simple linear model, Figure 7 shows that the model does a poor job at predicting the ob-

servations (red dots) and there is a large degree of uncertainty (shaded red ribbon) around the model-implied trajectory (solid red line). Now let's allow the model to learn from every 5th observation to supplement the predictions it makes using the model estimates alone. When the model is being estimated, it uses every data point for prediction, but visuals such as this can help provide the intuition for how the model works (note the updates to the prediction trajectory when new data is observed) and also provide useful checks on the model, in case predictions diverge substantially.

```
ctPredict(fit= f_basic, plot = TRUE, subjects = 6, removeObs = 5)
```

Figure 8

ctsem Predictions for Subject 6 Conditional on Parameter Estimates and Every 5th Observation.

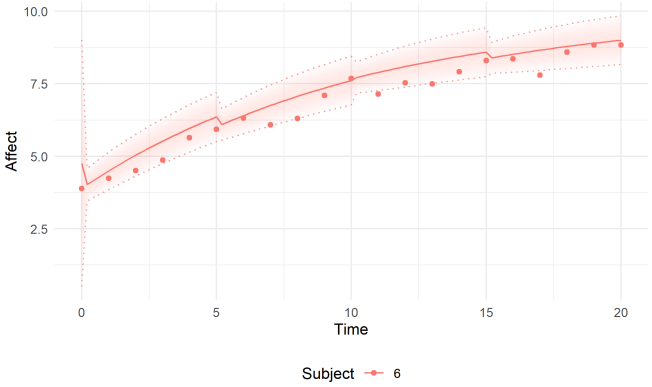


Figure 8 shows that the model already does a better job at predicting the observations, and the uncertainty in its predicted trajectory is reduced (though growing larger again towards the end of the time period before a new observation is fed into the model). To show actual predictive performance, the `removeObs` argument could simply be removed or set to `FALSE`.

Model Evaluation and Comparison

Having introduced continuous-time modelling, we now demonstrate how to evaluate and compare different model structures when we do not know the structure of our observed data. In empirical settings, this can be a very difficult and fascinating problem that requires deep inquiry, and we cannot cover all aspects and nuance here. To get people started with model evaluation, we demonstrate some tools and approaches that can help determine whether a model provides a good representation of the data, and whether it is better than any competing explanations (models) we may have.

There is no automatic rule for choosing a dynamic model. Model choice should triangulate theory, the relevant time scale, data density, identifiability, model-implied behavior, diagnostics, and parsimony. A more flexible model will often improve within-sample summaries, but this does not mean the extra parameters are interpretable or supported. Conversely, a model that is too simple may leave distinct sources of variation bundled into one parameter. The examples below therefore use R^2-style summaries only as one descriptive starting point, and then focus on prediction plots, residual autocorrelation, and posterior-predictive-style checks.

Let us imagine that after all the model building we have done, we still think that the best representation of an affect process is a linear trajectory. We will fit a linear model to our generated data from Section: State-Dependent Change with Fluctuations, but of course normally the true model is not known!

Fitting the Wrong Model: Linear Growth

We now specify and fit our linear growth model, with no state-dependency and no system noise, in `ctsem`.

```
m_linear <- ctModel( #define the ctsem model
  manifestNames = "Affect", #names of observed variables in data
  latentNames = "Affect", #names of latent processes
  time = 'Time', #name of time column in data
  id = 'Subject', #name of subject column in data
  type='ct', #continuous time
  MANIFESTVAR = 'residualSD', #sd of residual / measurement error
  LAMBDA = matrix(1,nrow=1,ncol=1), #factor loading matrix
  MANIFESTMEANS=0, #no measurement intercept
  CINT='B||FALSE', #continuous intercept with *no* random effects
  TOMEANS='t0Affect||TRUE', #initial affect with random effects
  DRIFT = 0,
  DIFFUSION = 0)
```

```
f_linear <- ctFit(datalong = data, model = m_linear)
```

A common form of model evaluation is to consider how much of the variance in the data has been explained by the model. This is typically quantified using the R^2 statistic (Nakagawa & Schielzeth, 2013) or some variation thereof. The R^2 tells us how much of the total variance in our data is explained by our model by dividing the residual (unexplained) variance ($V\hat{y}$) by the total variance(Vy) and subtracting this from 1. Because in `ctsem` models the total variance is not necessarily constant across individuals or time, this is computed using standardized residuals, and reported (as standardized residual variance / covariance) at the top of the ctsem summary output. This should not be treated as a stand-alone model selection criterion: complex models can improve within-sample explained variance while still implying the wrong process (Cross validation approaches, in which not all data are used for parameter estimation at once, can offer more robust model selection criteria, but we will not address those in detail beyond pointing the interested reader to the `ctLOO` function in `ctsem`).

```
s=summary(f_linear) #store summary of the fit
print(s$residCovStd) #print standardized residual (co)variance
```

```
       Affect
Affect  0.236
```

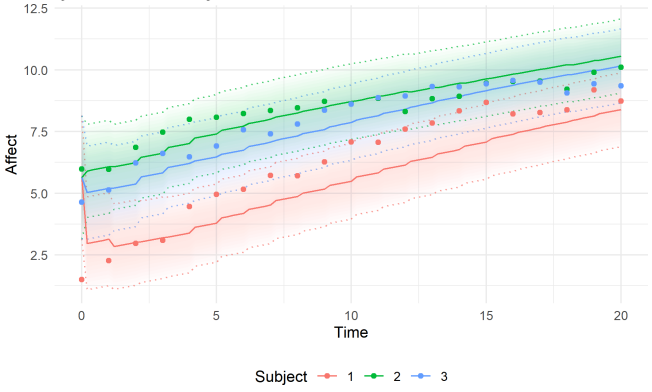
Now we can use the standardized residual variance estimate (also accessible as `s$residCovStd`, and equivalent to 1 - R^2) to see how much of the variance in our empirical data our model has accounted for. We can see that our model explains a high proportion of the variance in our observations. This is fairly typical for repeated measures data, and doesn't imply that our model is appropriate, but provides a useful baseline for comparison.

Let's begin considering model appropriateness by comparing the model's forward predictions against the observed data for a few subjects, shown in Figure 9. In this figure, lines show predictions conditional on parameter estimates and earlier data, and dots show observed values.

```
ctPredict(f_linear,plot=TRUE,subjects=1:3,
 kalmanvec = c('y','yprior')) #y=observed, yprior=predictions
```

Figure 9

Predictions from Misspecified Linear Model Versus Observed Data for Three Subjects.



A first question that might arise regarding Figure 9, is why aren't the model predictions the simple linear trajectory we specified? This occurs because as the model steps through time and 'sees' more data, it learns the individual difference parameters for each subject, and thus the predictions adjust and do not follow the simple linear trajectory. If we plotted the 'smoothed' predictions (conditional on all data, not just earlier data, use `ysmooth` instead of `yprior`) they would follow the linear trajectory. The forward predictions we see here, conditional only on parameter estimates and past data, are what the likelihood calculations and residuals are based on. If using an appropriate model, the residuals here — the difference between the observed and predicted data — should be entirely *unpredictable* based on earlier data or residuals.

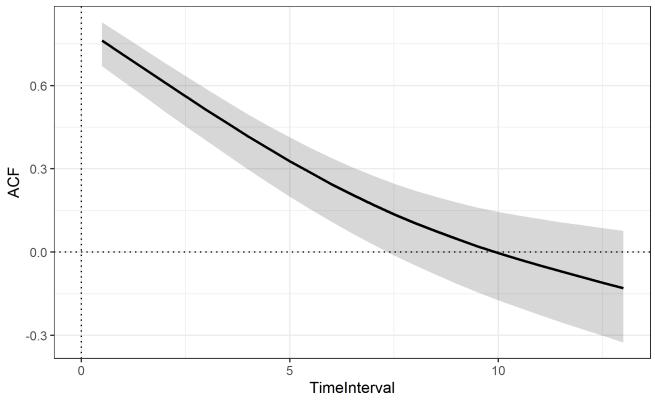
While far from a formal test, a visual inspection of Figure 9 suggests that this is *not* the case, as we can see some clear structure in the observed data that is not being captured by the model. This is visible in that when the model prediction is too low at one observation, it is also likely to be too low at the next observation — our residuals are *autocorrelated*. Autocorrelated residuals indicate that our model leaves some predictable structure unexplained (Gelman et al., 1996) and we could modify our model to improve the predictions. This state of affairs is broadly referred to as *model misspecification*. The predictions may still be useful for some purposes, but we should be extremely careful about interpretations of the model parameters or other generalisations.

An approach to examine all residuals at once in a more comprehensive manner is to check their autocorrelation. We can create a plot of the autocorrelation between residuals to visualize whether our model is missing any systematic patterns in our data. To do this we use the `ctACFresiduals` function.

```
ctACFresiduals(f_linear) #plot autocorrelation of residuals
```

Figure 10

Autocorrelation of Residuals from the Misspecified Linear Model.

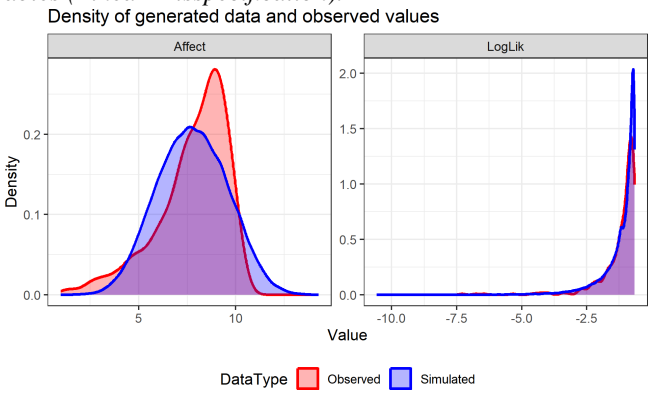


The autocorrelation in Figure 10 shows that there is a high autocorrelation between residuals, about 0.6 between temporally adjacent residuals. So now we know two things: (1) Our model explains a substantial amount of the variance in our data, (2) there is still room for improvement. The natural follow-up question is what should we improve? To get an idea of model misspecification we can generate data based on the parameter estimates of our model and plot those against our observed data. This approach is analogous to posterior predictive checks in a Bayesian framework, but can be applied in non-Bayesian contexts as well (Gelman & Shalizi, 2013).

```
f_linear <- ctGenerateFromFit(f_linear, nsamples=200,
  fullposterior = F, cores = 2) #add generated data to fit object
postpred<-ctPostPredPlots(f_linear) #create plots
print(postpred[[1]]) #show first plot --- overall density
```

Figure 11

Posterior Predictive Check: Density of Likelihood and Variables (Linear Misspecification).

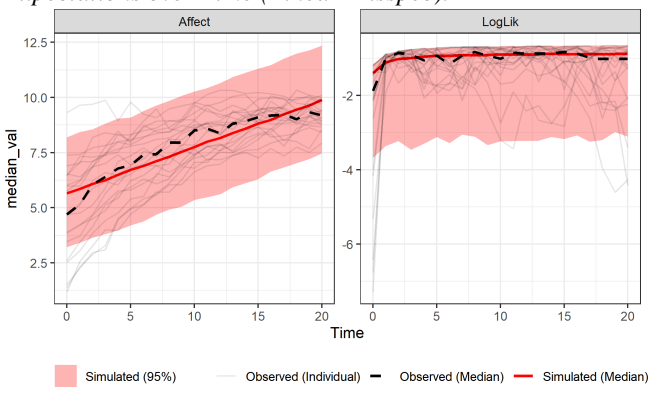


The density-based posterior predictive check in Figure 11 compares the empirical distribution of the observed variables and likelihood contributions with distributions generated from the fitted model. It is difficult to get a clear sense of exactly where things are imperfect from this aggregate view, but it provides another approach to see that there clearly are some patterns in the data that deviate from the model's expectations. Perfect overlap between the empirical and model-implied distributions for each variable doesn't necessarily mean the model is appropriate, but does at least indicate that the overall distributions are similar.

```
print(postpred[[2]]) #show second plot --- expectations over time
```

Figure 12

Posterior Predictive Check: Model-Implied Versus Empirical Expectations over Time (Linear Misspec).



The trajectory-based posterior predictive check in Figure 12 shows the model implied expectations (red) versus the empirical data (black) over time. Here it is quite clear that we are fitting a linear model to a nonlinear trajectory. Our model also seems to underestimate the variance at earlier time points while overestimating the variance at later observations. But how can we improve our model to better account for these observations?

Iterative Model Improvement

Taking a closer look at the observed individual growth trajectories in Figure 12 (fainter black lines), we can see there seems to be a dependency between people's initial affect values and their rate of change in affect. That is, people who start with very high affect values have an almost flat growth curve, while people with a low initial level have a very steep growth curve which slowly plateaus as they reach higher values of affect. Thus, it seems that we should add a state-dependency parameter to our model to capture this relationship. In empirical applications, this step should be justified by theory and diagnostics together: state dependence is most defensible when it is plausible that the current state at least partially determines subsequent change.

```
m_statedep <- ctModel(
  manifestNames = "Affect", #observed variables
  latentNames = "Affect", #latent processes
  time = 'Time', #time column
  id = 'Subject', #subject column
  type='ct', #continuous time
  MANIFESTVAR = 'residualSD', #residual variance
  LAMBDA = matrix(1,nrow=1,ncol=1), #factor loading matrix
  MANIFESTMEANS=0, #no measurement intercept
  CINT='B||FALSE', #continuous intercept with *no* random effects
  TOMEANS='t0Affect||TRUE', #initial affect with random effects
  DRIFT = 'stateDependence', #state-dependence
  DIFFUSION = 0) #no system noise
```

```
f_statedep <- ctFit(datalong = data, model = m_statedep)
```

```
s=summary(f_statedep) #store summary of the fit
print(s$residCovStd) #print standardized residual (co)variance
```

```
       Affect
Affect  0.116
```

Now we can again see if adding this state-dependence term has helped us explain more variance in our data by considering the R^2 — and indeed, we can see that we have around 10% residual variance, or 90% explained variation in our data — noticeably more than before. This is encouraging, but it is not sufficient: we also need to check whether the model-implied behavior and residual structure are more appropriate. Next, we inspect Figure 13 to see whether there is still information left in the residuals that our model could explain.

```
ctACFresiduals(f_statedep) #plot autocorrelation of residuals
```

Figure 13

Autocorrelation of Residuals from the State-Dependence Model.

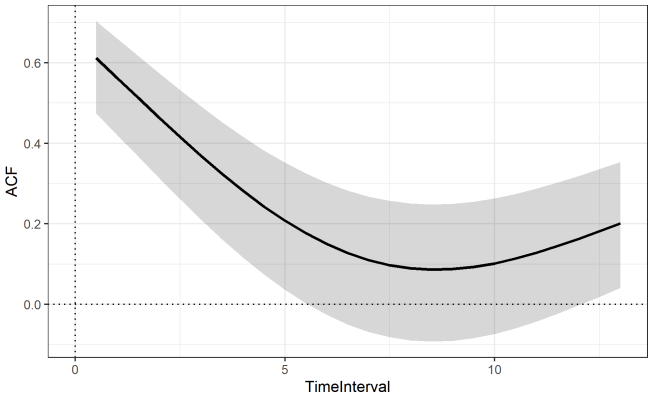
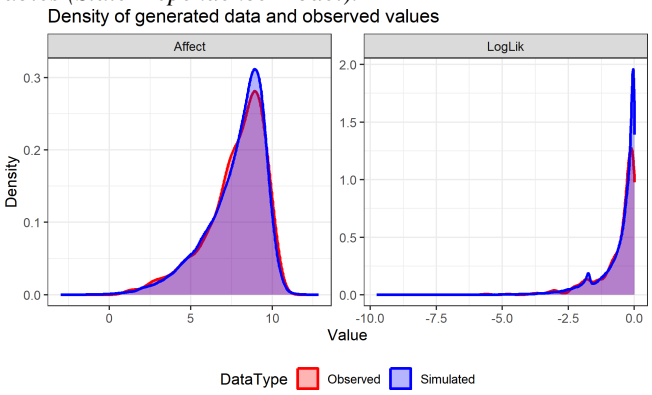


Figure 13 shows that the autocorrelation has also dropped substantially, but the residuals are still autocorrelated. This is telling us that there is yet more predictable variation that our model can leverage. So let's try to find out what it is by plotting the model generated data against our observed data.

```
fit <- ctGenerateFromFit(f_statedep, nsamples=200,
  fullposterior = F, cores = 2) #add generated data to fit object
postpred <- ctPostPredPlots(f_statedep) #create plots
print(postpred[[1]]) #show first plot --- overall density
```

Figure 14

Posterior Predictive Check: Density of Likelihood and Variables (State-Dependence Model).

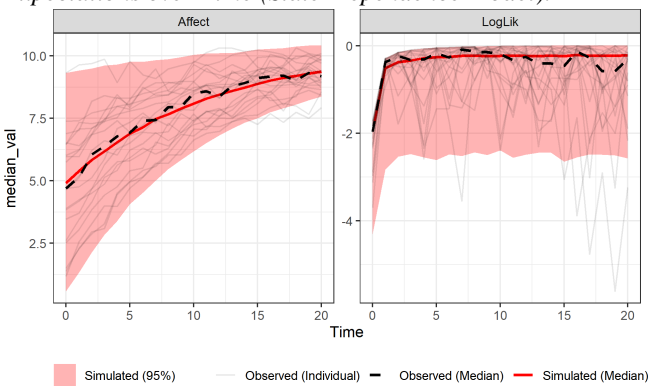


The density-based check in Figure 14 shows that the model implied distributions are now similar to the empirical distributions.

```
print(postpred[[2]]) #show second plot --- expectations over time
```

Figure 15

Posterior Predictive Check: Model-Implied Versus Empirical Expectations over Time (State-Dependence Model).

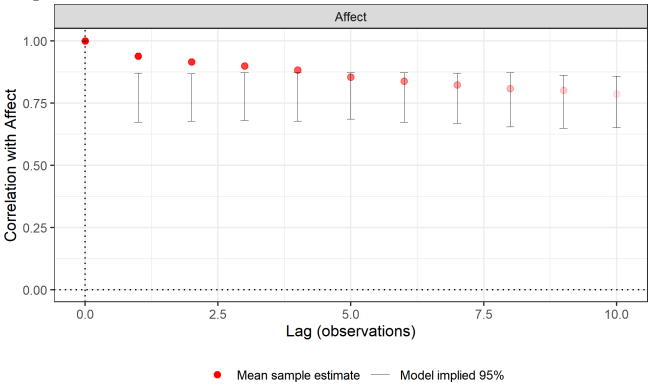


The trajectory-based check in Figure 15 shows that our model implied trajectory almost perfectly fits our observed trajectory, and the variance now also reduces over time in line with the observed trajectories converging to a similar level. So, we know there appears to be some predictable variation that our model could better account for, but it's difficult to determine how to do this. Examining empirical versus model implied lagged correlations can provide more insight:

```
ctFitCovCheck(f_statedep,plot = TRUE,cor=TRUE)
```

Figure 16

Empirical Versus Model-Implied Lagged Correlations (State-Dependence Model).



The lagged-correlation check in Figure 16 shows the empirical versus model implied lagged correlations for each variable (in a multivariate scenario, we would also see cross-correlations). We can see that the model implied correlations between one time point and later time points are stable, while the empirical correlations decrease over time. This makes sense because although we've now specified a state dependence parameter in the model, causing growth to slow and stabilise over time, we didn't allow for any *new* sources

of variation to be introduced into the process — the same variation that was there in the beginning persists on and on, smaller in magnitude, but equally correlated with earlier time points. Introducing a system noise term to our model will allow for new sources of variation to be introduced into the process, and we can see how this changes the lagged correlations. Substantively, this step is warranted when the theory and diagnostics suggest that unmodelled influences affect the latent process itself.

```
m_sysnoise <- ctModel(
  manifestNames = "Affect", #observed variables
  latentNames = "Affect", #latent processes
  time = 'Time', #time column
  id = 'Subject', #subject column
  type='ct', #continuous time
  MANIFESTVAR = 'residualSD', #residual variance
  LAMBDA = matrix(1,nrow=1,ncol=1), #factor loading matrix
  MANIFESTMEANS=0, #no measurement intercept
  CINT='B||FALSE', #continuous intercept with *no* random effects
  TOMEANS='tOAffect||TRUE', #initial affect with random effects
  DRIFT = 'stateDependence', #state-dependence
  DIFFUSION = 'systemNoise') #system noise
```

```
f_sysnoise <- ctFit(datalong = data, model = m_sysnoise)
```

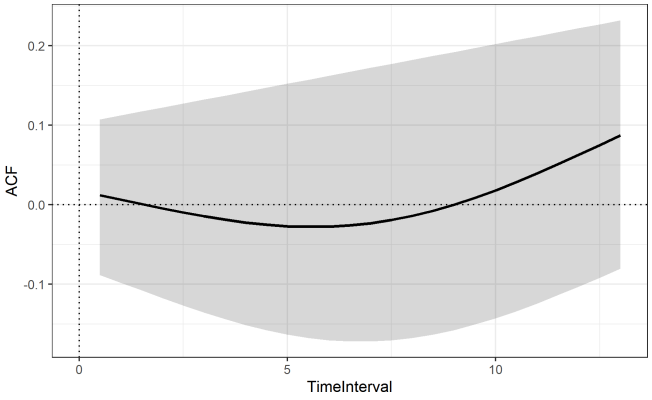
```
s=summary(f_sysnoise) #store summary of the fit
print(s$residCovStd) #print standardized residual (co)variance
```

```
       Affect
Affect  0.096
```

```
ctACFresiduals(f_sysnoise) #plot autocorrelation of residuals
```

Figure 17

Autocorrelation of Residuals from the Model with System Noise.



Overall, we can see that fitting the model to the data it generated leads to a greater R^2, near-zero autocorrelation between residuals in Figure 17, and model-implied lagged correlations that map well onto the observed lagged correlations.

In empirical contexts, this process is going to be more complicated — individual differences will likely need more serious consideration, and very often it will be difficult to obtain a model that appears perfect while also retaining some level of interpretability. Sensitivity checks, wherein core inferences are checked across a range of model specifications, can help to assess the robustness of the inferences to different model specifications (The example in Driver (2025) demonstrates this approach in the context of dynamic systems modelling).

Practical Troubleshooting

Dynamic state-space models can fail in ways that are informative. Common warning signs include Hessian warnings after fitting or extremely large or small standard errors / uncertainty intervals, unstable estimates across small changes in specification, parameter estimates at boundary values, implausibly fast or slow dynamics, and poor posterior predictive behavior. These problems can arise from the data, the model, or their mismatch. For example, variables with little or no variance, sparse observations, low reliability, an unsuitable time unit, or too many random effects can all make estimation fragile.

Several practical checks are useful if encountering a difficult fit. First, inspect the data scale and the time scale. At least within ctsem, models are often easier to estimate when time units of roughly 0.1 to 1.0 represent moderate amounts of change; very large or very tiny intervals can lead to numerical problems and or start values that are too extreme in one direction or the other. Centering or scaling observed variables can also help make start values and unconstrained parameter scales more reasonable. Scaling of covariates is particularly important — this is not handled by the software automatically. Time dependent predictors should be zero except when some effect occurs at a specific time. Time independent predictors should in most cases be centered to have mean zero, such that resulting parameter estimates are interpretable with respect to average values on the predictor variable. They should also have roughly variance of one, though this is not always necessary. A second check is to inspect the expanded model specification to ensure that default settings have not introduced unintended free parameters or random effects. Third, fit simpler intermediate models and add complexity only when the previous model behaves sensibly.

Finally, treat warnings as prompts for model criticism rather than as purely technical obstacles. A Hessian warning, boundary estimate, or sensitivity to specification changes may indicate that the data do not adequately identify the model, that the time scale is poorly chosen, or that a theoretically distinct influence is being forced into the wrong parameter because of misspecification. The appropriate response may be to rescale, simplify, collect denser or more reliable data, or revise the substantive model.

Modelling Multivariate Systems

Until now, we have looked at Alex's affect in isolation. But Alex is going to couples therapy, so how their partner,

Robin, feels probably has a pronounced impact on how Alex feels. Similarly, Alex's affect likely has a reciprocal impact on Robin's affect. This creates an interdependence betwen Alex and Robin's processes — a fundamental concept in dynamic systems modelling where multiple processes influence and are influenced by each other, creating a complex system of mutual dependencies. This can apply to a broad array of contexts: for example, health symptoms may influence one another, physiological and behavioral processes may be coupled, and parent-child or therapist-client systems may involve reciprocal dynamics. With additional processes in the system, we can model their interdependence in two fundamental ways:

1. We can introduce *a cross-effect state dependence term*, allowing Alex's affect at a given moment to directly impact how Robin's affect changes. This is essentially the state dependence we modeled within Alex's affect process, but now the state (affect level) that drives the rate of change is also coming from a different dynamic process – Robin's affect. We can also introduce the equivalent term for Robin.

2. *Correlation in system noise*, some of the random life events that cause fluctuations in Alex's affect trajectory are also likely to impact Robin's affect (and vice versa). We can model factors that simultaneously shift our couple's rate of change by allowing random noise terms for each partner, and allowing them to correlate — capturing the idea that Alex and Robin each experience unique influences, but also some shared influences. Such shared influences could include a household stressor, a conflict, a shared positive event, or another unobserved environmental input that affects both partners.

Cross Dynamics with Self and Partner Effects

A revised model which includes Alex and Robin's affect dynamics and their interdependence can be written with the same general equation as earlier, with only the dimensions of the equation components differing, e.g. the A matrix is now a 2x2 matrix. Alternatively, we can split the matrix equation two separate equations, one for each partner:

$$d\eta_1 = \big(A_1\eta_1(t) + A_{12}\eta_2(t) + B_1\big)dt + G_{11}dW_1(t) + G_{12}dW_2(t)$$

$$d\eta_2 = (A_2\eta_2(t) + A_{21}\eta_1(t) + B_2)dt + G_{21}dW_1(t) + G_{22}dW_2(t)$$

The equation for $d\eta_1$ (rate of change of Alex's affect), implies that Alex's rate of change depends on (1) their own current affect level ($A_1\eta_1(t)$ — the auto-effect), (2) their partner Robin's current affect level ($A_{12}\eta_2(t)$ — the cross-effect from partner), (3) a constant input (B_1), and (4) random noise that may affect both partners ($G_{11}dW_1(t) + G_{12}dW_2(t)$). In general, we would not be able to estimate G_{12} distinctly from G_{21} and vice versa, as they are correlated, so when it comes to estimation we will estimate a single parameter to represent the correlation between them, as well as the standard deviations of the noise for each partner. The equation for the rate of change of Robin's affect involves the same idea. The cross-effects (A_{12} and A_{21}) capture how partners directly influence each other's rate of change. If A_{12} is positive, when Robin's affect is high, Alex's affect is positively influenced — it either rises faster or declines slower.

Simulation

To demonstrate multivariate dynamics we generate data for 20 couples, building on our earlier univariate data generation. Each of the 40 people has their own initial level of affect, but all dynamics and growth terms are fixed across individuals for now. We first need to add an equation to represent the affect process of the additional partner, along with cross-effect and correlated-noise terms. In the code chunk below, there is now an equation for each partner's affect, named `Affect1` (for partner 1) and `Affect2` (for partner 2). Each equation has *two* new elements to capture interdependency in affect dynamics. (1) The change in affect for each person depends on their partner's level of affect at the same time point. The coefficient of this cross-effect is denoted by `Across` (e.g. `dAffect1 <- A*Affect1 + Across * Affect2 + B`). (2) We allow the random fluctuations in the couple's affect process to covary. We do this by generating correlated noise according to a specified `DIFFUSIONcov` covariance matrix, where the diagonal elements specify the variance for each process and the off-diagonal elements specify the covariance between processes.

```
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect1 <- rnorm(n = NSubjects, mean = 5, sd = 2)
t0Affect2 <- rnorm(n = NSubjects, mean = 5, sd = 2)
A <- -.2 #continuous time state dependence
Across <- .1 #cross-effect state dependence
B <- 1 #continuous intercept
#covariance matrix for system noise
DIFFUSIONcov <- matrix(c(1, .05, .05, .1), nrow=2, ncol=2)

data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect1 = rep(NA,NSubjects*Nobs),
  Affect2 = rep(NA,NSubjects*Nobs)) #now with two individuals

Nsteps <- 100 #steps between observations
row <- 0 # initialize row counter
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1 #increment row counter
    if(obsi==1){ #set initial affect
      Affect1 <- t0Affect1[subi]
      Affect2 <- t0Affect2[subi]
    }
    if(obsi>1){ #update state via small steps in time
      for(stepi in 1:Nsteps){ #for each step
        # compute deterministic slopes of affect
        dAffect1 <- A*Affect1 + Across * Affect2 + B
        dAffect2 <- A*Affect2 + Across * Affect1 + B
```

```
        # generate correlated noise
        systemNoise <- MASS::mvrnorm(n=1, mu=c(0,0),
          Sigma=DIFFUSIONcov/Nsteps) #scaled by stepsize

        #update state using slopes and time step + noise
        Affect1 <- Affect1 + dAffect1/Nsteps + systemNoise[1]
        Affect2 <- Affect2 + dAffect2/Nsteps + systemNoise[2]
      }
    }
    data$Affect1[row] <- Affect1 #input affect data
    data$Affect2[row] <- Affect2 #input affect data
    data$Time[row] <- times[obsi] #input time data
    data$Subject[row] <- subi #input subject data
  }
}
#add measurement error
data$Affect1 <- data$Affect1+rnorm(n=nrow(data),mean = 0,sd=.2)
data$Affect2 <- data$Affect2+rnorm(n=nrow(data),mean = 0,sd=.2)
```

Model Specification

The model syntax for `ctsem` for the bivariate model, which could also be called a continuous-time cross-lagged panel model, is:

```
m_couple <- ctModel( #define the ctsem model
  manifestNames = c("Affect1",'Affect2'), #observed variables
  latentNames = c("Affect1",'Affect2'), #names of latent processes
  time = 'Time', #name of time column in dataset
  id = 'Subject', #name of subject column in dataset
  type='ct', #use continuous time (dt for discrete-time)
  MANIFESTVAR = c( #covariance matrix of measurement error
    'residualSD1',0, #sd on diagonal, 0 correlation assumed.
    0, 'residualSD2'),
  LAMBDA = diag(1,2), #diagonal factor loading matrix (identity)
  MANIFESTMEANS=c(0,0), #zero measurement intercept / offset
  CINT=c('B1||F','B2||F'), #continuous intercept no random effects
  TOMEANS=c('t0Affect1||TRUE',
  't0Affect2||TRUE'), #initial affect with random effects
  DRIFT = c(
    'auto1','cross12', #auto effect partner 1, cross-effect 2 to 1
    'cross21','auto2'), #cross-effect 1 to 2, auto effect for 2
  DIFFUSION = c(
    'systemNoise1', 0, #system noise for 1, 0 in upper triangle
    'systemNoiseCor', 'systemNoise2')) #corr, and sd for 2
```

Checking and understanding model specification for multivariate systems can beeasier when looking at the model equations in matrix form — we can generate a LaTeX formatted representation of the equations by running the `ctModelLatex()` function.[1] The model output for the multivariate model is shown in Figure 18 (created with some additional display arguments via `ctModelLatex(m_couple, splitDynamics=F,splitMeasurement=F, includeNote=F)`, which brings together the different elements of the process and measurement equations, as well as individual differences.

The first equations in Figure 18 show the initial state and (when appropriate) subject specific parameter distribution, confirming that we have two parameters in our model that vary across individuals, `t0Affect1` and `t0Affect2`, and these are assumed normally distributed according to some estimated mean and covariance, *possibly* with some transformation applied after this distribution to ensure parameters remain within necessary bounds (primarily relevant when e.g., correlation or standard deviation parameters are estimated, see the ctsem manual for more details on this).

Next we look at the dynamic model component of Figure 18, made up of deterministic and random change components. On the left, the vector $(d\text{Affect}_1, d\text{Affect}_2)$ represents the rates of change for both partners. On the right, the drift matrix shows how each partner's current affect influences each partner's rate of change. The diagonal elements (A_1, A_2) are the auto-effects (how each person's own affect influences their own change), while off-diagonal elements (A_{12}, A_{21}) are the cross-effects (how one partner affects the other's change). The CINT (continuous intercept) vector contains the constants (B_1, B_2) that are steady inputs for each partner.

Turning to the random process change in Figure 18, it is created by a complex function (UcorSDtoChol) that takes the $DIFFUSION$ matrix as input, where the diagonal parameters are standard deviations and the lower-triangle off-diagonal parameters represent 'unconstrained correlations' between the noise terms. Our standard deviation parameters are `systemNoise1` and `systemNoise2` and the correlation parameter is called `systemNoiseCor`. These unconstrained correlation parameters are unfortunately complex to interpret directly, but necessary to allow for continuous-moderation and random-effects (for interpretation, obtaining the DIFFUSION covariance from the **System Matrices** section of `summary()` or from `ctSummaryMatrices()` is recommended, possibly with a conversion to a correlation matrix via the `cov2cor()` function).

The latent process model is linked to observations using the observation equation in Figure 18. This equation also comprises a deterministic component and a random component, and links what we actually observe ($\text{Affect}_1(t)$, $\text{Affect}_2(t)$) to the underlying latent processes ($\text{Affect}_1(t)$, $\text{Affect}_2(t)$) that represent 'true' affect.

The deterministic component of the measurement model is given by $\Lambda\eta(t) + \tau$, where Λ is a matrix of regression weights used to link the latent process to our observations, and is here an identity matrix (1s on diagonal, 0s off-diagonal), meaning each observed affect measure directly reflects its corresponding latent affect with a 1-to-1 relationship (no scaling or transformation). The MANIFESTMEANS τ reflects a constant that can be used to shift the relationship between observations and a latent variable, but here is a zero vector, meaning there's no systematic bias or offset in measurements (This becomes particularly useful when estimating factor models, but can also be estimated instead of `CINT`, such that latent processes converge towards zero and `MANIFESTMEANS` offsets this an appropriate amount for the data).

The random component of the measurement model (observation noise) is given by $\epsilon(t)$, where ϵ is a multivariate

[1] The `tinytex` package may be needed if no latex compiler is available. Install `tinytex` in R via `install.packages('tinytex')`, then run `tinytex::install_tinytex()`. In case of errors along the way, restarting R / Rstudio and clearing the workspace often helps.

Figure 18

Matrix Equations for the Bivariate ctsem Model.

$$\text{Initial and subject parameter distribution:} \quad \underbrace{\begin{bmatrix}\text{t0Affect1}_i\\ \text{t0Affect2}_i\end{bmatrix}}_{\phi(i)} \sim \text{tform}\left\{\text{N}\left(\begin{bmatrix}\text{t0Affect1}\\ \text{t0Affect2}\end{bmatrix}, \begin{bmatrix}\text{rawPCov\_1\_1} & \text{rawPCov\_2\_1}\\ \text{rawPCov\_2\_1} & \text{rawPCov\_2\_2}\end{bmatrix}\right)\right\}$$

$$\text{Dynamics:} \quad \underbrace{\text{d}\begin{bmatrix}\text{Affect1}\\ \text{Affect2}\end{bmatrix}(t)}_{\text{d}\boldsymbol{\eta}(t)} = \left(\underbrace{\begin{bmatrix}\text{auto1} & \text{cross12}\\ \text{cross21} & \text{auto2}\end{bmatrix}}_{\mathbf{A}\ \text{DRIFT}} \underbrace{\begin{bmatrix}\text{Affect1}\\ \text{Affect2}\end{bmatrix}(t)}_{\boldsymbol{\eta}(t)} + \underbrace{\begin{bmatrix}\text{B1}\\ \text{B2}\end{bmatrix}}_{\mathbf{b}\ \text{CINT}}\right)\text{dt} \quad + \underbrace{UcorSDtoChol\left\{\begin{bmatrix}\text{systemNoise1} & 0\\ \text{systemNoiseCor} & \text{systemNoise2}\end{bmatrix}\right\}}_{\mathbf{G}\ \text{DIFFUSION}} \underbrace{\text{d}\begin{bmatrix}W_1\\ W_2\end{bmatrix}(t)}_{\text{d}\mathbf{W}(t)}$$

$$\text{Measurement:} \quad \underbrace{\begin{bmatrix}\text{Affect1}\\ \text{Affect2}\end{bmatrix}(t)}_{\mathbf{Y}(t)} = \underbrace{\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}}_{\boldsymbol{\Lambda}\ \text{LAMBDA}} \underbrace{\begin{bmatrix}\text{Affect1}\\ \text{Affect2}\end{bmatrix}(t)}_{\boldsymbol{\eta}(t)} + \underbrace{\begin{bmatrix}0\\ 0\end{bmatrix}}_{\boldsymbol{\tau}\ \text{MANIFESTMEANS}} \quad + \underbrace{UcorSDtoChol\left\{\begin{bmatrix}\text{residualSD1} & 0\\ 0 & \text{residualSD2}\end{bmatrix}\right\}}_{\boldsymbol{\Theta}\ \text{MANIFESTVAR}} \underbrace{\begin{bmatrix}\epsilon_1\\ \epsilon_2\end{bmatrix}(t)}_{\boldsymbol{\epsilon}(t)}$$

$$\text{System noise distribution per time step:} \quad \Delta\left[W_{j\in[1,2]}\right](t-u) \sim \text{N}(0, t-u) \qquad \text{Observation noise distribution:} \quad \left[\epsilon_{j\in[1,2]}\right](t) \sim \text{N}(0,1)$$

normal distribution with mean 0 and (usually diagonal) covariance matrix Θ. The Θ matrix (MANIFESTVAR) contains measurement error standard deviations (residualSD1, residualSD2) which scale the disturbance terms ϵ, and off-diagonal elements (unconstrained correlations) are here set to 0.

When specifying matrices in `ctsem`, they can be created using the `matrix()` function in R, but can also be passed in as a vector (except for the LAMBDA, factor-loading matrix) using the `c()` function, which will be read in **row-wise** in contrast to typical R convention.

Model Fitting

Now we fit our model and could, as earlier, use `summary` to view estimated parameters — here we just demonstrate one way to obtain more easily interpretable correlation / covariance matrices from the output.

```
f_couple <- ctFit(datalong = data, model = m_couple)
parmatrices <- ctSummaryMatrices(f_couple) #estimated matrices
print(cov2cor(parmatrices$DIFFUSIONcov)) #system noise correlation
```

```
          Affect1   Affect2
Affect1 1.0000000 0.1561811
Affect2 0.1561811 1.0000000
```

Visualize Predictions

This time, we'll look at how well the model predicts our observations using every earlier data point (i.e., just as the model does it) for two dyads:

```
ctPredict(fit = f_couple, plot=TRUE, subjects = 1:2,
  kalmanvec=c('y','yprior'), facets='Subject')
```

Figure 19

Bivariate Model Predictions for Dyads 1 and 2, Conditional on Earlier Data Points.

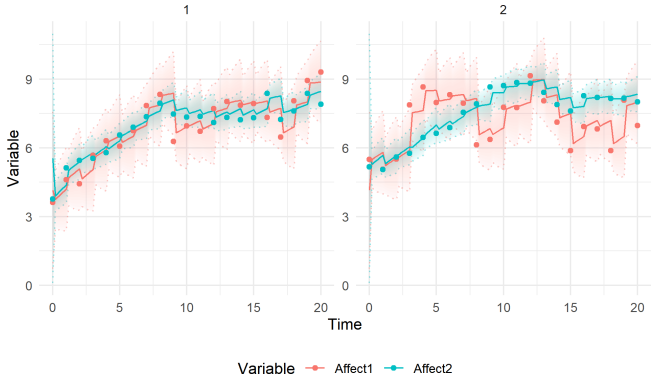


Figure 19 shows predictions for observed values of the four individuals in dyads 1 (left) and 2 (right). The uncertainty (shaded polygons) reflects both the system noise and measurement error – we could examine the latent process predictions instead by changing the `kalmanvec` argument to `'etaprior'` instead of `'yprior'`.

Visualize Dynamics — Independent Shocks

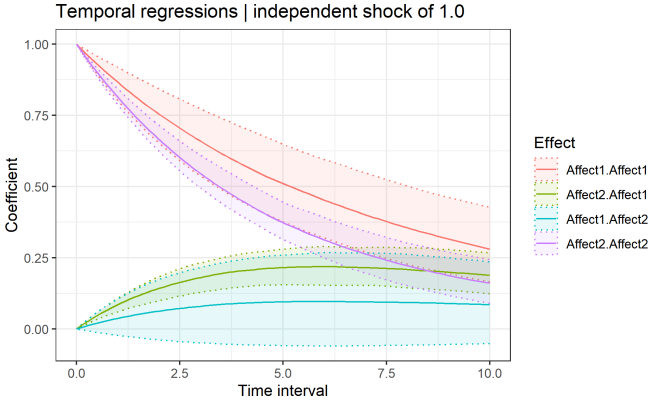
We can visualize the model-implied dynamics (i.e.,auto and cross-effects) using an 'impulse response function' that shows us the outcome of a hypothetical experiment. In this experiment, we intervene on each of our processes one by one, shifting them up by 1.0 at time 0 without affecting any other processes in the model at that point in time, then plot

what happens to the processes as time passes after this intervention. Examining Figure 20, we see that if we increase Alex's affect by 1 at time 0, it is expected to still be approximately 0.40 above baseline after 5 weeks (red solid line, `Affect1.Affect1` at time interval = 5) because of self-dependency and the bidirectional cross-effects between Alex and Robin. Robin's affect however doesn't jump initially in response to this intervention on Alex, but rises gradually in response to Alex's higher affect — it is expected to be 0.20 by week 5 (green solid line, `Affect2.Affect1` in Figure 20 at time interval = 5). We can also visualize what happens if we increase Robin's affect by 1 *instead of Alex's* by viewing the blue and purple solid lines. In this case, where our hypothetical impulse generates a change of 1.0, the plotted values also correspond directly to the discrete-time coefficients implied by the model, for the times shown — so with an interval of 5 weeks, Alex's affect exhibits an autoregression of 0.40, and so on.

```
ctDiscretePars(f_couple,plot=T)
```

Figure 20

Impulse Response Functions Assuming Independent Shocks



Visualize Dynamics — Correlated Shocks

Our hypothetical experiment with independent shocks may not generalize well to real world dynamic systems estimated from observational data. This is because often when some shock happens to one process, other processes in the system also tend to be affected — even though we may model this correlation in the system noise, we have only been able to estimate model parameters in this correlated context, and extrapolating to the hypothetical independent shock context is often problematic (see Driver, 2023, for more on the combined interpretation of system noise and dynamics).

One way to consider this issue is to consider the correlation in the system noise, which tells us to what extent Alex and Robin's fluctuations are related, and incorporate this into the initial impulse — depicting essentially a more 'typical' impulse for the system. Instead of reflecting a hypothetical experiment, such an impulse instead tells us how the model predicts processes to change after a 'typical' shock, which does not just affect one process. For example, if the system noise correlation is estimated at 0.25, then when Alex's affect increases by 1 unit due to a random shock, Robin's affect is expected to also change by approximately 0.25 units. This gives a more realistic depiction of the dynamic interplay between the partners.

```
ctDiscretePars(f_couple,plot=T, observational=TRUE)
```

Figure 21

Impulse Response Function Assuming Correlated Shocks

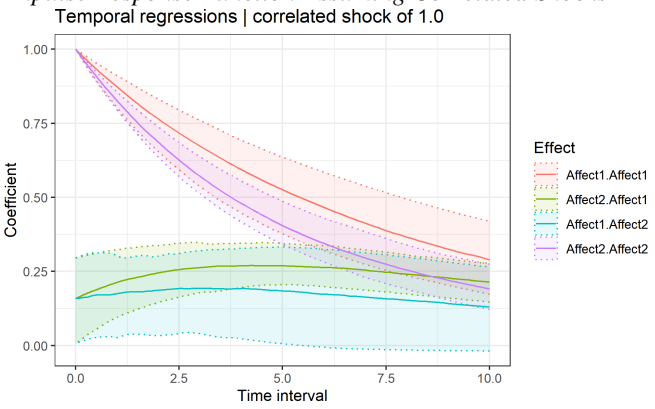


Figure 21 shows that when Alex's affect increases by 1 at T0, Robin's affect is expected to contemporaneously fluctuate in the same direction as well (positive system noise correlation). The positive cross-effect coefficients (more easily seen in the summary output or Figure 20) also show a positive bidirectional relationship between the two partners, such that when one partner's affect increases we can expect a subsequent increase in the other partner's affect. Because we generated the data, we can be sure that this cross-effect reflects a genuine causal effect, but in a real-world scenario, we must consider the possibility of confounding variables that could explain this estimated relationship. The system noise correlation can account for confounders that change very quickly (compared to the speed of change of affect), but is unlikely to sufficiently account for confounders that change at a similar speed to affect. For instance, a sudden loud noise that startles both partners will lead to a very brief fluctuation in affect for both partners, which could be reasonably reflected in the system noise. However, a prolonged period of gloomy weather might cause a gradual systematic shift in the affect of both partners that could lead to estimated cross-effects that a) appear substantial and important in statistical terms but b) do not reflect any causal effect between Alex and Robin. Such confounders are not accounted for in this model and indeed are very difficult to account for in general, but allowing for stable

differences (in e.g., the continuous intercept) is one way that very slow / stable confounders can be (partially) accounted for — more on this in Section: Individual Differences in System Dynamics.

Individual Differences in System Dynamics

In the models so far, we have assumed that all individuals have the same system dynamics, and may only differ in the start and end points of their trajectories. In reality, people may have highly heterogeneous system dynamics. This is because there are likely to be an immense number of factors that contribute to what we might consider as the individual's dynamic system. Factors that change slowly or not at all, with respect to our observed time window, can reasonably be treated as stable individual differences. There might be individual differences in the magnitude of random fluctuations, with some people having more stable systems while others are more volatile. Alternatively, coupling effects between partners of a dyad (or all manner of other cognitive or behavioural process) may be stronger for some people than for others — perhaps some people are more dependent on a partner than others, or stress affects cognitive performance more for some people.

Cross Dynamics with Individual Differences and Time-Independent Moderation

Stable moderators are useful whenever people or systems may differ in their dynamic parameters because of relatively enduring characteristics. In other psychological applications, these moderators might be diagnostic group, age, trait affect, baseline stress, treatment condition, relationship length, socioeconomic context, or a stable environmental exposure. A key question is whether the moderator plausibly changes the dynamic process or measurement model itself, rather than merely predicting average levels.

We consider two sources of individual differences. First, we assume that individuals have differences in their long-term, post therapy baseline affect. This will be done by allowing individual differences in the continuous intercept, which makes our model a continuous-time analogue to the popular random intercept cross-lagged-panel model (Hamaker et al., 2015). Second, we assume that couples who have been together longer have a stronger influence on each others' affect. This will be included via individual differences in the strength of the cross-effect state-dependence terms, and by allowing the size of each couple's cross-effect to depend on the average time they spend together.

Simulation

Building on the data generation code from Section: Cross Dynamics with Self and Partner Effects, we now also sample two continuous intercepts per dyad from a normal distribution, `B1 <- rnorm(n = NSubjects, mean = 2, sd = .3)` (also `B2`). We also assume that partners are somewhat similar in affect tendencies, thus we add a common value sampled from a normal distribution to both. That is, `Bcommon <- rnorm(n = NSubjects, mean = 1, sd = .2)` is added to the sampled values of `B1` and `B2`.

We also sample a cross-effect for each couple from a normal distribution and then add an effect of time the couple has spent together `Across <- rnorm(n= NSubjects, mean = .1, sd=.05) + .1*TogetherZ`. For simplicity in this example, we assume that the cross-effect is the same for both partners in the dyad.

```
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect1 <- rnorm(n = NSubjects, mean = 5, sd = 2)
t0Affect2 <- rnorm(n = NSubjects, mean = 5, sd = 2)
Together <- runif(n=NSubjects, min=0, max=20) #years together
TogetherZ <- scale(Together) #standardise time together
A <- -.4 #continuous time state dependence
DIFFUSIONcov <- matrix(c(.1, .05, .05, .1), nrow=2, ncol=2)
Across <- rnorm(n= NSubjects, mean = .1, sd=.05) + .1*TogetherZ

# generate continuous intercepts for each individual
Bcommon <- rnorm(n = NSubjects, mean = 1, sd = .2)
B1 <- Bcommon + rnorm(n = NSubjects, mean = 2, sd = .3) #partner 1
B2 <- Bcommon + rnorm(n = NSubjects, mean = 2, sd = .3) #partner 2

data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect1 = rep(NA,NSubjects*Nobs),
  Affect2 = rep(NA,NSubjects*Nobs)) #affect for two individuals

Nsteps <- 100 # steps between each observation
row <- 0 #init current row tracker
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1
    if(obsi==1){ # if first time point, set to initial affect
      Affect1 <- t0Affect1[subi]
      Affect2 <- t0Affect2[subi]
    }
    if(obsi>1){ # else update state via small steps in time
      for(stepi in 1:Nsteps){ # for each step
        #compute deterministic slopes
        dAffect1 <- A*Affect1 + Across[subi] * Affect2 + B1[subi]
        dAffect2 <- A*Affect2 + Across[subi] * Affect1 + B2[subi]
        # generate correlated noise scaled by stepsize
        systemNoise <- MASS::mvrnorm(n=1, mu=c(0,0),
          Sigma=DIFFUSIONcov/Nsteps)
        #update states using slope and time step, add noise
        Affect1<- Affect1 + dAffect1 * 1/Nsteps+systemNoise[1]
        Affect2<- Affect2 + dAffect2 * 1/Nsteps+systemNoise[2]
      }
    }
    data$Affect1[row] <- Affect1 #input affect data
    data$Affect2[row] <- Affect2 #input affect data
    data$Time[row] <- times[obsi] #input time data
    data$Subject[row] <- subi #input subject data
    data$TogetherZ[row] <- TogetherZ[subi] #input time together
  }
}
# add measurement error
data$Affect1 <- data$Affect1+rnorm(n=nrow(data),mean=0,sd=.2)
data$Affect2 <- data$Affect2+rnorm(n=nrow(data),mean=0,sd=.2)

data$Dyad <- factor(data$Subject) #prep data for plotting
plot_trajectory(data[data$Subject %in% c(1,2,3),],y="Affect1",
  show_legend=TRUE, y2="Affect2",color_var="Dyad")
```

Figure 22

Affect Trajectories with Stable Individual Differences.

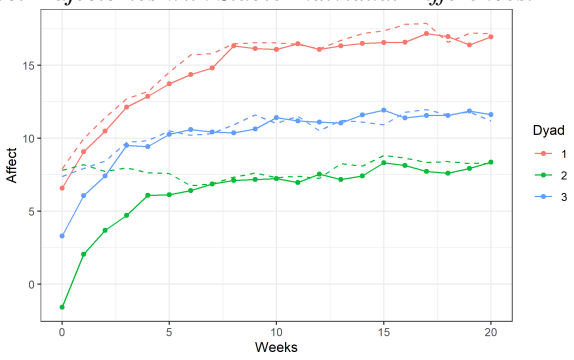


Figure 22 shows the affect trajectories for three generated dyads, with stable individual differences – the similarity of partners in each dyad is likely quite unrealistic, but serves for our example.

Model Fitting

Stable individual differences can be accommodated using both 'fixed' and 'random' effect approaches. In the fixed effect approach, we rely on observed covariates (time-independent predictors in `ctsem`) to moderate the system parameters and explain the individual differences. In `ctsem` these are restricted to linear effects on the internal 'raw' or 'unconstrained' parameters (parameters such as correlations or standard deviations have non-linear transforms applied afterwards, to ensure they cannot go out of bounds — see the ctsem manual for more detail).

In the random effect approach, we assume that the individual differences are at least partly due to unobserved factors, and estimate the variance and correlations in these factors, relying purely on the observed data (without covariates) to estimate where each individual's parameter(s) sit with respect to the population distribution.

By combining fixed and random effects, we try to improve the estimated model for each individual by allowing individuals to have unique system dynamics, but not 'completely unique' — we still rely to some extent on how other individuals behave (for more on continuous-time hierarchical modelling see Driver & Voelkle, 2018a). In `ctsem` the complexities of random-effects specification matrices in the state-space approach are abstracted away from the user, but they may be manually added if additional control is needed, via the specification of additional latent processes (for details state-expansion approach see the Driver & Tomasik, 2023). For our example, we will leave the specification of the random effects to the `ctsem` back-end, which estimates all random-effects as correlated.

Our `ctsem` model specification now needs to include the time couples spent together (`TogetherZ`) as a stable source of individual differences (time-independent predictor). This is done using the argument `TIpredNames = c('TogetherZ')`. By default when covariates are included in `ctsem` they moderate all parameters of the system. We turn the default moderation off using `tipredDefault = FALSE`. Because we want our time-independent predictor to moderate a specific parameter, we can explicitly specify this moderation by inserting the name of the predictor after any free parameter labels we wish to moderate, distinguishing between different 'slots' of the parameter specification using the pipe `|` operator. Random-effects specifications are added by putting `TRUE` in the third slot, and time-independent predictors in the fifth slot. Multiple moderators could be specified by separating them with commas. The parameter specification for the cross-effect state dependence is thus `cross21||TRUE||TogetherZ`. It is important to note that moderators in `ctsem` can dramatically influence the interpretation of parameters and lead to confusion if care is not taken — usually centering, and possibly scaling, the moderator(s) is a good idea. Since we also want random effects for the continuous intercept of each partner we specify these using `CINT=c('B1||TRUE','B2||TRUE')`. More details on the parameter specification slots can be found in the `ctsem` manual (https://cran.r-project.org/package=ctsem/vignettes/hierarchicalmanual.pdf).

```
m_indiff <- ctModel( #define the ctsem model
  tipredDefault = FALSE, #moderation disabled unless specified
  TIpredNames = c('TogetherZ'), #time independent predictors
  manifestNames = c("Affect1",'Affect2'), #observed variables
  latentNames = c("Affect1",'Affect2'), #names of latent processes
  time = 'Time', #name of time column in dataset
  id = 'Subject', #name of subject column in dataset
  type='ct', #use continuous time
  MANIFESTVAR = c('residualSD1',0,
    0, 'residualSD2'), #sd's of the residual / measurement error
  LAMBDA = diag(1,2), #factor loading matrix
  MANIFESTMEANS=c(0,0), #no measurement intercept / offset
  CINT=c('B1||TRUE','B2||TRUE'), #cint with random effects
  TOMEANS=c('t0Affect1||TRUE','t0Affect2||TRUE'), #initial affect
  DRIFT = c( #auto and cross effects matrix, cross effects with...
    'auto1', 'cross21||TRUE||TogetherZ', #random effects and...
    'cross21||TRUE||TogetherZ','auto2' ), #moderation
  DIFFUSION = c( #system noise matrix
    'systemNoise1', 0, #sd for subj 1, 0 in upper triangle
    'systemNoiseCor', 'systemNoise2')) #correlation, sd for subj 2
```

```
f_indiff <- ctFit(datalong = data, model = m_indiff)
```

```
s = summary(f_indiff) #store summary of the fit
```

Now if we look at the Time-independent predictor effects section of the summary (`s$tipreds`), we have an estimated value for the effect of time together on the cross-effect state dependence. We can see that the cross-effect state dependence increases by approximately 0.09 for each increase of 1 in standardized time together. Thus, partner's who have spent more time together tend to have more influence on each other's affect.

```
print(s$tipreds)
```

```
                       mean    sd  2.5%   50% 97.5%      z
tip_TogetherZ_cross21 0.094 0.008 0.079 0.094 0.111 11.273
```

The Random-effects correlations section of the summary (`s$rawpopcorr`) shows the estimated correlations between the random effects of the model — the initial states and the continuous intercepts. The correlation between the random effects for the continuous intercepts (the `B2__B1` parameter) is approximately -0.02.

```
print(round(s$rawpopcorr,2))
```

```
                     mean   sd  2.5%   50% 97.5%     z
t0Affect2__t0Affect1 0.12 0.20 -0.28  0.12  0.51  0.57
cross21__t0Affect1   0.31 0.26 -0.26  0.34  0.75  1.22
B1__t0Affect1        0.39 0.27 -0.24  0.44  0.78  1.42
B2__t0Affect1       -0.01 0.35 -0.65 -0.01  0.68 -0.03
cross21__t0Affect2  -0.33 0.28 -0.76 -0.38  0.35 -1.17
B1__t0Affect2       -0.14 0.28 -0.64 -0.14  0.40 -0.51
B2__t0Affect2       -0.04 0.36 -0.69 -0.03  0.60 -0.12
B1__cross21          0.36 0.36 -0.51  0.46  0.83  1.00
B2__cross21         -0.44 0.30 -0.86 -0.50  0.26 -1.48
B2__B1              -0.02 0.53 -0.84 -0.10  0.92 -0.04
```

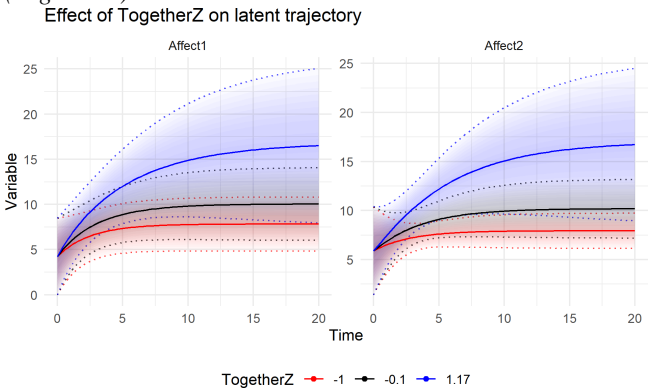
Visualize Individual Differences

Viewing model implied trajectories and dynamics, *conditional* on certain values of moderators, is a very useful way to understand moderated models.We can do this in `ctsem` using the `ctPredictTIP` function, which creates two sets of plots. The first, Figure 23, shows model-implied latent affect trajectories conditional on +/-1 standard deviation and the median of time spent together (`TogetherZ`). Couples that spent more time together have a faster rate of growth in affect and also tend to plateau at a higher affect value:

```
tipredplots <- ctPredictTIP(f_indiff, tipreds = c('TogetherZ'))
print(tipredplots$Process$Latent) #latent process predictions
```

Figure 23

Predicted Latent Affect Trajectories by Time Spent Together (TogetherZ).

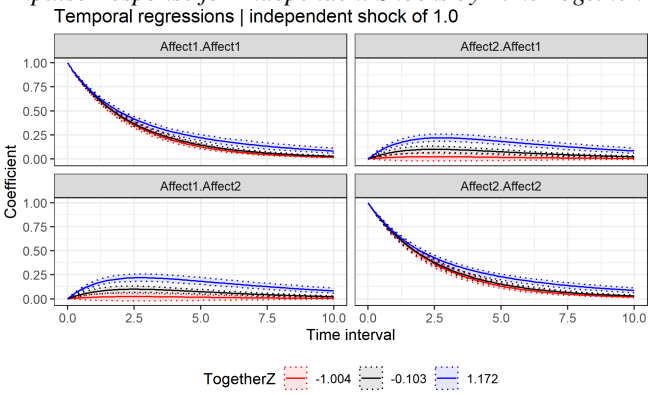


The second set of plots created by `ctPredictTIP`, in Figure 24, shows how the model-implied impulse response functions differ by time spent together. When a person experiences a shock to their affect (increase of 1), their partner is expected to change more over time if they have spent more time together. The four facets represent the two auto effects for each partner and the two cross effects (indexed in standard row-column order, with the column as the cause). We visualise the independent shock type here, reflecting what our estimated model suggests should happen under an intervention to one or the other affect process.

```
print(tipredplots$Dynamics$Independent$TogetherZ)
```

Figure 24

Impulse Response for Independent Shocks by Time Together.



Interventions and External Events

Many research questions require explicit representation of external events or interventions. One fundamental form models an event as an instantaneous impulse to a latent process; the effect then propagates and dissipates according to the existing system dynamics. Many influences on psychological processes vary within persons over time and can be reasonably represented as such impulses. For example, therapy sessions, stressful life events, medication doses, or social interactions could all be represented as impulses, in `ctsem` these are called *time-dependent predictors* (TDpreds).

Sometimes dissipation through the existing system dynamics is insufficient to represent the external input in question. This could occur when the initial impulse better represents the *start* of some process that exerts influence, such as drinking a coffee, or changing school. Both these cases can be reasonably defined by a specific event at a particular time, but coffee takes some time to digest and effects will rise and fall in the body according to a specific dynamic, while a change in school might often be better represented as some new influence that arises and doesn't dissipate, or dissipates slowly. An external input can develop through a separate latent process

with its own dynamics, allowing intervention effects that develop differently from the main system and can therefore generate persistent shifts rather than transient impulses — if combined with the moderation techniques described in Section: Latent State Moderation of System Parameters, this approach can be used to represent a wide range of intervention effects where the intervention alters the system dynamics rather than simply the levels of the latent processes.

Impulse Effects

Consider Alex, who attends weekly therapy sessions during their 20-week treatment period. Suppose that Alex really feels good from the social contact and support of therapy, and each session provides a temporary boost to their affect. The longer term effect of the therapy is already accounted for by the growth trajectory from earlier examples. Between sessions, the short term effect of therapy fades away and their affect gradually returns toward their general growth trajectory. The key insight is that we do not need to separately model the "decay" of the therapy effect — it emerges naturally from the state dependence dynamics we have already specified.

Mathematically, if a therapy session occurs at time t_k with effect size M, the latent state receives an instantaneous impulse:

$$\eta(t_k^+) = \eta(t_k^-) + M$$

where $\eta(t_k^-)$ is the state just before the session and $\eta(t_k^+)$ is the state just after.

Simulation

For this example we revert back to the univariate case with a single latent process (i.e., just Alex's affect). To simulate data with therapy sessions as impulse effects, we create a time-dependent predictor `Therapy` that equals 1 at observations when a therapy session occurs and 0 otherwise. For simplicity, we schedule therapy sessions at regular intervals (every 3 time units).

```
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)
A <- -.2 #continuous time state dependence
B <- rnorm(n = NSubjects, mean = 1, sd = .3) #CINT
G <- .3 #system noise coefficient
M <- 1.5 #therapy impulse effect (positive = boost to affect)

# Define therapy sessions (every 3 time units, starting at time 3)
therapyTimes <- seq(from=3, to=max(times), by=3)

#create empty data.frame to fill step by step
data <- data.frame(Subject = rep(NA, NSubjects*Nobs),
  Time = rep(NA, NSubjects*Nobs),
  Affect = rep(NA, NSubjects*Nobs),
  Therapy = rep(NA, NSubjects*Nobs))

Nsteps <- 10 # steps between observations
row <- 0 #initialize row counter
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){
    row <- row + 1
    currentTime <- times[obsi]
    therapyToday <- ifelse(currentTime %in% therapyTimes, 1, 0)

    if(obsi == 1) Affect <- t0Affect[subi]
    if(obsi > 1){ #update state via small steps in time
      for(stepi in 1:Nsteps){
        dAffect <- A * Affect + B[subi] #deterministic slope
        #update state using slope and time step
        Affect <- Affect + dAffect * 1/Nsteps +
          G * rnorm(n=1, mean=0, sd=sqrt(1/Nsteps)) #system noise
      }
    }
    #add impulse if therapy session (dummy variable)
    Affect <- Affect + M * therapyToday
    data$Affect[row] <- Affect #add affect to data
    data$Time[row] <- currentTime #add time to data
    data$Subject[row] <- subi #add subject to data
    data$Therapy[row] <- therapyToday #add therapy to data
  }
}
#add measurement error (after creating process!)
data$Affect <- data$Affect+rnorm(n=nrow(data), mean = 0, sd = .2)
```

Model Fitting

To fit this model in `ctsem`, we add the `Therapy` variable as a time-dependent predictor using the `TDpredNames` argument. By default, `ctsem` estimates the effect of time-dependent predictors on each latent process, here we explicitly name the parameter `td_Affect_Therapy`. The estimated coefficient represents the effect of the impulse(s) on the latent state. We do not estimate individual differences in this effect, but such differences could be of interest if going further with the model.

```
m_therapy <- ctModel(
  manifestNames = "Affect",
  latentNames = "Affect",
  TDpredNames = "Therapy", #time-dependent predictor
  TDPREDEFFECT = c('td_Affect_Therapy'),
  time = 'Time',
  id = 'Subject',
  type = 'ct',
  MANIFESTVAR = 'residualSD',
  LAMBDA = matrix(1, nrow=1, ncol=1),
  MANIFESTMEANS = 0,
  CINT = 'B||TRUE',
  TOMEANS = 't0Affect||TRUE',
  DRIFT = 'stateDependence',
  DIFFUSION = 'systemNoise')
```

```
f_therapy <- ctFit(datalong = data, model = m_therapy)
```

```
s <- summary(f_therapy)
print(s$popmeans)
```

```
                    mean    sd   2.5%    50%  97.5%
t0Affect           4.619 0.395  3.875  4.625  5.406
stateDependence   -0.216 0.015 -0.247 -0.216 -0.187
systemNoise        0.295 0.027  0.243  0.294  0.350
residualSD         0.225 0.021  0.185  0.224  0.269
td_Affect_Therapy  1.591 0.048  1.497  1.591  1.691
B                  1.098 0.129  0.843  1.103  1.338
```

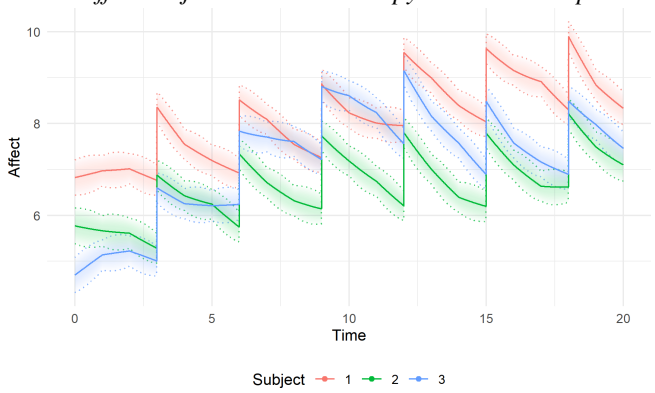
The `td_Affect_Therapy` parameter in the Fixed-effects / Population means section represents the estimated impulse effect of therapy sessions. A positive value indicates that therapy boosts affect. Our simulated value was `M = 1.5`, and the estimate is 1.59.

We can use the `ctPredict` function to plot the latent affect trajectories with therapy sessions as impulses. In Figure 25 we plot the estimated latent processes, conditional on past, present, and future data for each subject ('smoothed' estimates), and also use a fine-grained `timestep` to make the vertical nature of the impulses clearer.

```
ctPredict(f_therapy,kalmanvec=c('etasmooth'),
  plot=TRUE,subjects=1:3,timestep=.01)
```

Figure 25

Latent Affect Trajectories with Therapy Sessions as Impulses.



Persistent Intervention

Here we consider a shock to the system that *does not* dissipate, where we allow the persistence, or 'shape' of the interventioneffect to differ from the rest of the system. This approach can be generalized to achieve things such as separated trends and dynamics processes (Driver & Tomasik, 2023), cross-effects that operate at different time scales (Haehner et al., 2025), oscillatory or more complex dynamic patterns (Voelkle & Oud, 2013), and more.

The impulse model in Section: Impulse Effects implies that the therapy effect is immediate and persists according to the system dynamics. This is probably the simplest approach to thinking about such effects, but in reality, the effects of an intervention or event may be more complex, and may depend on the individual, the context, may induce other changes in the system, and may persist or transfer through the system in different ways to the typical system behavior. For example, while the regular ups and downs of life may dissipate relatively quickly, a traumatic event may have direct effects that last over a long period and may require more consideration.

Instead of relying on the more 'general' system parameters to capture the long term trajectory in response to couples therapy sessions — remembering that from the beginning of this tutorial we have always worked with a rising affect over time — we can instead explicitly model this rise, and maintenance at a new level, as a function of therapy sessions. For more examples of intervention effects and how to specify them in ctsem we refer the reader to (Driver & Voelkle, 2018b). A major benefit of this approach is that, for example, patients may have very different intervention schedules — allowing the trend to develop based on the specific schedule, rather than a general assumption of a specific form, can be more realistic and flexible. Similar persistent-input structures can represent other psychological processes such as cumulative practice, treatment dose, habit formation, sustained exposure, or gradual learning.

To implement this approach, we need to specify a latent process which is impacted by our observed intervention variable, and is *not measured by any variables*. This process is used to represent the hypothetical development of the intervention effect over time. To try to make this explicit:

Simple impulse form: Intervention directly impacts affect process, dissipation governed by system dynamics.

Separated form: Intervention affects a separate latent process that is independent of the main system dynamics, with either estimated or fixed dynamic parameters — this process in turn then impacts the affect process.

This latent intervention process can have its own autoeffect parameter which allows us to specify an accelerating, dissipating, or stable effect across time. If the state dependence is set to 0, the latent intervention process will remain stable (e.g., remaining at 1 after an initial upwards shock of 1) and thereby generate a permanent shift in the downstream affect process over time. If the state dependence is set to a negative value (e.g. -0.1), then the effect of the intervention will dissipate over time, but not as quickly as it would have in the pure impulse form.

Simulation

In our data generating block, we include an input variable `M` which directly influences the state of our new latent intervention process `TherapyAcc` (accumulated therapy). The rate of change in the latent intervention process `dTherapyAcc` is given by the previous value of `TherapyAcc` multiplied by a state dependence coefficient, `dTherapyAcc <- AM*TherapyAcc`. Because the state dependence coefficient is 0 (autoregression = 1) in the case of a persistent input effect, the rate of change is always 0. At the time of therapy sessions the input effect `M` is added directly to `TherapyAcc` — there is no need to track this via the rate of change, as it is an instantaneous impulse.

Between therapy sessions, because the state dependence parameter is zero, the latent intervention process `TherapyAcc` will not change; the therapy sessions generate a step-like process, gradually accumulating over time as therapy sessions occur (one might argue that it should probably also slowly decay over time). We nevertheless show the code for updating the latent intervention process with a small step in time, to illustrate the process.

To model the impact of our latent intervention pro-

cess on our affect process, we make the rate of change in affect dependent on the state of the intervention process at the current moment (`dAffect <- A*Affect + B + TherapyAcc`) — this is just another cross-effect in our drift matrix, from the matrix equation perspective.

```
NSubjects <- 20
times <- seq(from=0, to=100, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect <- rnorm(n = NSubjects, mean = 5, sd = 2)
A <- -.1 #continuous time state dependence
B <- 0.5 #continuous intercept reduced, growth via therapy effect
G <- .1 #system noise coefficient
M <- 0.2 #input variable
AM <- 0 #latent input process autoregression

# randomly sample 3 time points for therapy for each subject
therapyTimes <- lapply(1:NSubjects, function(x){
  return(sample(times, size = 3, replace = FALSE))
})

data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect = rep(NA,NSubjects*Nobs))

Nsteps <- 100 # steps between each observation
row <- 0 #init current row tracker
for(subi in 1:NSubjects){#for each subject
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1 # increment row counter
    #create therapy dummy:
    Therapy <- as.integer(times[obsi] %in% therapyTimes[[subi]])
    if(obsi==1){ #if first time point
      Affect <- t0Affect[subi] #set initial affect
      TherapyAcc <- 0 # initialize latent input process
    }
    if(obsi>1){ #otherwise update state via small steps in time
      for(stepi in 1:Nsteps){ #for each step
        # compute deterministic slopes
        dTherapyAcc <- AM*TherapyAcc # therapy slope update
        dAffect <- A*Affect+B+TherapyAcc #affect slope update
        #update states using slope and time step, add noise
        TherapyAcc <- TherapyAcc + dTherapyAcc/Nsteps
        Affect <- Affect + dAffect/Nsteps +
         G * rnorm(n=1, mean=0, sd=sqrt(1/Nsteps)) #system noise
      }
    }
    TherapyAcc <- TherapyAcc + M*Therapy # effect of therapy
    #store data:
    data$Affect[row] <- Affect #input affect data
    data$Time[row] <- times[obsi] #input time data
    data$Subject[row] <- subi #input subject data
    data$Therapy[row] <- Therapy #input therapy dummy
    data$TherapyAcc[row] <- TherapyAcc #input intervention process
  }
}
#add measurement error
data$Affect<-data$Affect+rnorm(n=nrow(data),mean=0,sd =.2)
```

Model Fitting

To fit this model we need to specify a latent intervention process that only reflects our therapy accumulation process, without any extra noise or baseline effects. To do this, we include the latent intervention process and set all the extra influences on it, that do not come directly from the observed intervention variable, to 0. Thus, we set to zero the random fluctuations (`DIFFUSION`), continuous intercept `CINT`, initial value (`TOMEANS`), initial variance (`TOVAR`). We have not had to deal with the `TOVAR` argument before — it is only relevant when there are no individual differences in the starting value of the process, which occurs either when the differences are disabled, or when the starting value is fixed, as in this case. The `TOVAR` matrix provides variance at the initial time point for each subject, reflecting uncertainty in the starting value. We do not want this variance, so we can specify a single zero (which will actually create an appropriate size matrix of zeros).

We then set the cross-effect from our latent intervention process on our affect process to 1 in `DRIFT[1,2]`. This specifies a direct effect of the current state of the latent intervention process on the rate of change in our affect process. Lastly, we set the effect of the observed intervention variable on our latent affect process to zero, and estimate its direct effect on our latent intervention process (`TDPREDEFFECT = c(0,"TherapyEffect")`). Thus, our observed intervention only directly impacts the latent intervention process, which in turn directly impacts our latent affect process. Instead of estimating the effect of Therapy on TherapyAcc, alternatively we could have fixed this to 1 and estimated the effect of TherapyAcc on Affect.

```
m_persist <- ctModel(
  manifestNames = "Affect", #observed variables
  latentNames = c("Affect","TherapyAcc"), #latent process names
  TDpredNames = 'Therapy', #time dependent predictors
  time = 'Time', #time column
  id = 'Subject', #subject column
  type='ct', #continuous time
  MANIFESTVAR = 'residualSD', #residual variance
  LAMBDA = matrix(1,nrow=1,ncol=2), #factor loading matrix
  MANIFESTMEANS=0, #no measurement intercept
  CINT=c('B||FALSE', #cint with *no* random effects for Affect,
    "0||FALSE"), # and no CINT at all for therapy accumulation
  TDPREDEFFECT = c(0,"TherapyEffect"), #Therapy on TherapyAcc
  TOMEANS=c("t0Affect||TRUE", #initial affect with random effects,
    "0||FALSE"), # and therapy accumulation starts at 0
  TOVAR=0, # no within-subject variance in starting values
  DRIFT = c( #auto effect and TherapyAcc effect on affect.
    'stateDependence', 1, #autoeffect
    0, 0), #therapy accumulation process is static
  DIFFUSION = c('systemNoise',0, #system noise for affect
    0, 0)) #no system noise for therapy accumulation
```

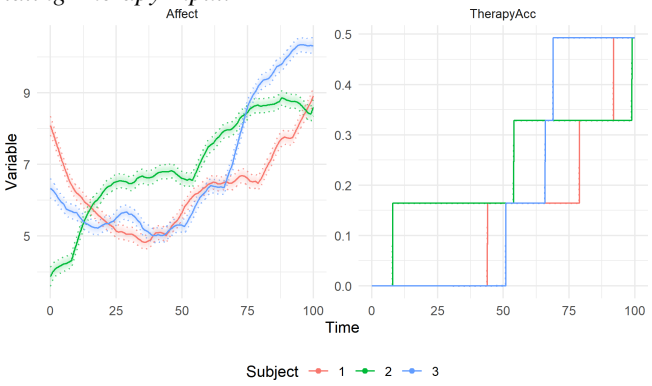
```
f_persist <- ctFit(datalong = data, model = m_persist)
```

We can plot the smoothed estimates of the latent variable states for the first three subjects in our dataset, which shows the increase in the accumulated therapy at the time of the intervention, which causes increases in affect over time. Figure 26 highlights the instantaneous impulse of the therapy session on accumulated therapy, and the long term growth in affect due to accumulated therapy.

```
ctPredict(f_persist, plot=TRUE, subjects=1:3, kalmanvec='etasmooth')
```

Figure 26

Smoothed Latent Processes for Three Subjects with Accumulating Therapy Input.



Time-Varying Parameters

The models considered so far assumed dynamic parameters are fixed over the observation window, but sometimes it is important to consider how system parameters themselves may change over time. One way to examine this is to allow a latent state to moderate system parameters — in the following example, when partner affect transmits more strongly at higher levels of activation. Other approaches might make system parameters a function of observed data such as a time-varying predictor, or just the time variable itself.

Latent State Moderation of System Parameters

In Section: Cross Dynamics with Individual Differences and Time-Independent Moderation, we allowed coupling to depend on how much time a couple spends together, treated as a stable between-couple characteristic. We may also want coupling itself to depend on the partners' current affective states. For example, one partner's affect may transmit more strongly to the other when that partner is already highly activated than when affect is low — a form of emotional contagion or amplification at higher levels of arousal.

Beyond couple dynamics, the same logic applies whenever the strength of one process's effect on another depends on the current level of a modeled latent state. Stress might increase symptom coupling only when arousal is already high; social support might buffer negative affect only below a certain level of distress; or cognitive performance might be more sensitive to physiological arousal at higher levels of activation. In each case, the moderator is not observed directly as a separate variable, but is represented by the current state of another process in the model. This approach is not limited to cross-effects: latent state moderation can in principle act on auto-effects, intercepts, or other system parameters.

Simulation

To generate data with latent state moderation of cross-effect coupling, we allow each partner's cross-effect to depend on the other partner's current affect. The cross-effect for partner 1 becomes `cross12 <- cross12base + cross12mod*inv_logit(Affect2/10)`, and symmetrically for partner 2. When `cross12mod` is positive, influence from one partner rises as the other partner's affect increases. We use the inverse logit on a scaled affect variable to ensure that change due to moderation is limited, such that extreme lows and extreme highs do not moderate substantially more than moderate lows and highs.

```
NSubjects <- 20
times <- seq(from=0, to=20, by=1) #time points of measurement
Nobs <- length(times) #number of observations per subject
t0Affect1 <- rnorm(n = NSubjects, mean = 5, sd = 2)
t0Affect2 <- rnorm(n = NSubjects, mean = 5, sd = 2)
A <- -.4 #continuous time state dependence
cross12base <- .1 #baseline cross-effect
cross12mod <- .08 #state moderation

# generate continuous intercepts for each individual
Bcommon <- rnorm(n = NSubjects, mean = 1, sd = .2)
B1 <- Bcommon + rnorm(n = NSubjects, mean = 2, sd = .3) #partner 1
B2 <- Bcommon + rnorm(n = NSubjects, mean = 2, sd = .3) #partner 2

# DIFFUSIONcov: covariance matrix for system noise
DIFFUSIONcov <- matrix(c(.1, .05, .05, .1), nrow=2, ncol=2)

data <- data.frame(Subject= rep(NA,NSubjects*Nobs),
  Time = rep(NA,NSubjects*Nobs),
  Affect1 = rep(NA,NSubjects*Nobs),
  Affect2 = rep(NA,NSubjects*Nobs))

Nsteps <- 100 #steps between each observation
row <- 0 #initialize row counter
for(subi in 1:NSubjects){
  for(obsi in 1:Nobs){ #for each observation of a subject
    row <- row + 1 #increment row counter
    if(obsi==1){ #set initial affect
      Affect1 <- t0Affect1[subi]
      Affect2 <- t0Affect2[subi]
    }
    if(obsi>1){ #update state via small steps in time
      for(stepi in 1:Nsteps){ #for each step
        #compute cross effects with latent state moderation
        cross12 <- cross12base + cross12mod*inv_logit(Affect2/10)
        cross21 <- cross12base + cross12mod*inv_logit(Affect1/10)
        #compute deterministic slopes
        dAffect1 <- A*Affect1 +
          cross12*Affect2 + B1[subi]
        dAffect2 <- A*Affect2 +
          cross21*Affect1 + B2[subi]
        # generate correlated noise scaled by stepsize
        systemNoise <- MASS::mvrnorm(n=1, mu=c(0,0),
          Sigma=DIFFUSIONcov/Nsteps)
        #update states using slope and time step, add noise
        Affect1 <- Affect1 + dAffect1/Nsteps + systemNoise[1]
        Affect2 <- Affect2 + dAffect2/Nsteps + systemNoise[2]
      }
    }
    data$Affect1[row] <- Affect1 #input affect data
    data$Affect2[row] <- Affect2 #input affect data
    data$Time[row] <- times[obsi] #input time data
    data$Subject[row] <- subi #input subject data
  }
}
# add measurement error
data$Affect1 <- data$Affect1 + rnorm(n=nrow(data),mean=0,sd = .2)
data$Affect2 <- data$Affect2 + rnorm(n=nrow(data),mean=0,sd = .2)
```

Model Fitting

In `ctsem`, we can moderate system parameters by specifying the system parameter (e.g., one of the cross-effects) as a *function*, rather than as a simple fixed value or free parameter. Such a function can depend on one or more free parameters, fixed values, mathematical functions, latent states, and time-dependent predictors. We write the expression in the appropriate matrix element, refer to the moderating latent process by name wherever needed, and have to tell `ctsem` which elements of the expression are free parameters to be estimated by specifying these in the `PARS` argument of `ctModel`.

For this example, we modify the cross-effects in the `DRIFT` matrix to be a function of the latent state of the other partner, as `cross12base + cross12mod*inv_logit(Affect2/10)` for the effect of partner 2 on partner 1, and `cross12base + cross12mod*inv_logit(Affect1/10)` for the reverse direction. We reduce the number of parameters by assuming the auto- and cross-effect parameters of each partner to be equal (noting that the effect at any moment will *not* be equal unless the latent states are equal), and declare the new free parameters in the `PARS` argument.

```
m_statemod <- ctModel(
  manifestNames = c("Affect1",'Affect2'), #observed variables
  latentNames = c("Affect1",'Affect2'), #names of latent processes
  time = 'Time', #name of time column in dataset
  id = 'Subject', #name of subject column in dataset
  type='ct', #use continuous time
  MANIFESTVAR = c('residualSD1',0,
    0, 'residualSD2'), #sd's of the residual / measurement error
  LAMBDA = diag(1,2), #factor loading matrix
  MANIFESTMEANS=c(0,0), #no measurement intercept / offset
  CINT=c('B1||TRUE','B2||TRUE'), #cint with random effects
  TOMEANS=c('t0Affect1||TRUE','t0Affect2||TRUE'), #initial affect
  DRIFT = c( #cross effects moderated by partner latent state
    'auto1', 'cross12base + cross12mod*inv_logit(Affect2/10)',
    'cross12base + cross12mod*inv_logit(Affect1/10)','auto1'),
  DIFFUSION = c( #system noise matrix
    'systemNoise1', 0, #sd for subj 1, 0 in upper triangle
    'systemNoiseCor', 'systemNoise2'),#correlation, sd for subj 2
    PARS=c('cross12base','cross12mod'))#extra params
```

```
f_statemod <- ctFit(datalong = data, model = m_statemod)
s_statemod <- summary(f_statemod)
```

Now we fit and summarise the results (in cases with matrix elements as expressions, the model may need to be compiled before fitting, which occurs automatically and usually takes a couple of minutes). The Fixed-effects section of summary shows that the `cross12base` parameter is estimated to be approximately 0.03, and the `cross12mod` parameter is estimated to be 0.15. A positive `cross12mod` means that partner influence strengthens as the partner's affect increases.

Further Works and Resources

Beyond the core ctsem workflow, several tutorials and reviews map the broader longitudinal modelling landscape and can help with the array of choices to be made in linking theory to models and data. For discrete-time foundations, a concise, practice-oriented overview of growth curve models addresses common pitfalls and implementation details (Curran et al., 2010). For intensive longitudinal data that blur boundaries between multilevel models, time series, and SEM, dynamic structural equation modelling (DSEM) provides an integrated framework with worked examples and diagnostics (Asparouhov et al., 2018). Bridging discrete- and continuous-time perspectives, step-by-step introductions show how cross-lagged and autoregressive panel models can be translated into their continuous-time counterparts with stochastic differential equations and estimated within SEM (Voelkle et al., 2012). For researchers using cross-lagged panel models specifically, some criticisms clarify interpretational limits and motivates model choices that better align statistical parameters with substantive within-person processes (Berry & Willoughby, 2017; Hamaker et al., 2015).

For dynamic-systems modelling that formalizes change mechanisms directly as differential (or stochastic differential) equations, there are other accessible software and methodological resources. The dynr package paper provides a hands-on introduction to specifying linear/nonlinear, discrete/continuous-time models with code recipes and estimation details (Ou et al., 2019). Methodological tutorials demonstrate hierarchical stochastic differential equation models for affect dynamics, highlighting how drift and diffusion parameters map to psychological regulation and noise (Oravecz et al., 2011), and how short and long term timescales may be considered in the one model (Boker et al., 2024).

Linking theory to model requires explicit mappings from theoretical constructs to parameters, along with clearly stated scope conditions and levels of analysis, some general guidelines of which can be found in Wilson and Collins (2019). A range of works explore how formal models can serve, inform, or be tested against theories, and advocates using formal models as a practical toolkit for theory construction (Haslbeck et al., 2022; Robinaugh et al., 2021). Recent proposals to integrate intra- and inter-individual phenomena make these mappings explicit, clarifying which parameters encode within-person dynamics versus between-person structure and how each bears on theoretical claims (Borsboom & Haslbeck, 2024). For panel designs, careful attention to time scale and confounding is crucial: cross-lagged effects are often misinterpreted when temporal assumptions are mismatched or when time-invariant confounders are not handled appropriately; guidance is available for diagnosing these issues and providing more defensible causal interpretations (Driver, 2025; Rohrer & Murayama, 2023). Finally, when theories posit continuous change, formalizing them as continuous-time systems can yield parameters (e.g., drift, diffusion) that align more directly with hypothesized mechanisms

and developmental relations (Driver & Tomasik, 2023).

There are many more extensions and model variants than we considered in this paper, but one follows very naturally from the models we have specified: nonlinear measurement models when the observed scale changes sensitivity across the latent range, such as we might see if for instance negative affect tends to change only slowly at lower levels and more rapidly at higher levels, leading to skewed distributions of the observed variable but a latent process that can still be modelled as continuous with Gaussian noise. This approach was used in the example in Driver (2025) and R code can be found in its supplement.

Conclusion

Modern computational tools and software packages such as `ctsem` simplify the process of building intricate statistical models to meet the surge of interest in intensive longitudinal data. However, the interpretation of complicated models remains challenging despite the increasing ease of their implementation. The central lesson of this tutorial is therefore not simply how to fit a particular software model, but how to move between theory, generative assumptions, simulation, estimation, model checking, and revision.

The running example focused on couples' affect dynamics, but the same modelling logic applies whenever psychological theory concerns change, regulation, coupling, inputs, or heterogeneity over time. State dependence can represent inertia or recovery; cross-effects can represent influence among people, symptoms, behaviors, or physiological processes; diffusion can represent unmodelled inputs to the latent process; measurement error can represent imperfect observation; and moderators can represent stable or time-varying conditions under which dynamics or measurement properties differ. These ideas are useful only to the extent that the model, data, and theory support the interpretation. A complex model that is weakly identified can mislead, but so can an overly simple model that bundles distinct mechanisms into one parameter.

The practical goal behind this tutorial is to provide the tools and intuition for iterative and cautious model building. Researchers should begin with explicit theoretical assumptions, simulate the implications of those assumptions where possible, fit models that are no more complex than the theory and data can support, check whether the model-implied behavior is plausible, and simplify or revise when the model fails. We hope this tutorial helps researchers use dynamic systems models as transparent tools for theory development rather than as black-box descriptions of longitudinal data.

References

Aalen, O. O., Røysland, K., Gran, J. M., Kouyos, R., & Lange, T. (2016). Can we believe the DAGs? A comment on the relationship between causal DAGs and mechanisms. *Statistical Methods in Medical Research*, *25*(5), 2294–2314. https://doi.org/10.1177/0962280213520436

Asparouhov, T., Hamaker, E. L., & Muthén, B. (2018). Dynamic Structural Equation Models. *Structural Equation Modeling: A Multidisciplinary Journal*, *25*(3), 359–388. https://doi.org/10.1080/10705511.2017.1406803

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Berry, D., & Willoughby, M. T. (2017). On the practical interpretability of cross-lagged panel models: Rethinking a developmental workhorse. *Child Development*, *88*(4), 1186–1206. https://doi.org/10.1111/cdev.12660

Boker, S. M., Daniel, K. E., & Orzek, J. (2024). Separating Long-Term Equilibrium Adaptation from Short-Term Self-Regulation Dynamics Using Latent Differential Equations. *Multivariate Behavioral Research*, *59*(6), 1177–1187. https://doi.org/10.1080/00273171.2023.2228302

Borsboom, D., & Haslbeck, J. (2024). Integrating Intra- and Interindividual Phenomena in Psychological Theories. *Multivariate Behavioral Research*, *59*(6), 1290–1309. https://doi.org/10.1080/00273171.2024.2336178

Borsboom, D., van der Maas, H. L. J., Dalege, J., Kievit, R. A., & Haig, B. D. (2021). Theory construction methodology: A practical framework for building theories in psychology. *Perspectives on Psychological Science*, *16*(4), 756–766. https://doi.org/10.1177/1745691620969647

Butler, E. A., & Randall, A. K. (2013). Emotional coregulation in close relationships. *Emotion Review*, *5*(2), 202–210. https://doi.org/10.1177/1754073912451630

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., & Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, *76*(1). https://doi.org/10.18637/jss.v076.i01

Curran, P. J., Obeidat, K., & Losardo, D. (2010). Twelve frequently asked questions about growth curve modeling. *Journal of Cognition and Development*, *11*(2), 121–136. https://doi.org/10.1080/15248371003699969

Driver, C. C. (2023). Inference with cross-lagged effects – common causes in dynamic systems. *PsyArXiv*. https://doi.org/10.31234/osf.io/y3e9d

Driver, C. C. (2025). Inference with cross-lagged effects—Problems in time. *Psychological Methods*, *30*(1), 174–202. https://doi.org/10.1037/met0000665

Driver, C. C., Oud, J. H. L., & Voelkle, M. C. (2017). Continuous Time Structural Equation Modeling with R package ctsem. *Journal of Statistical Software*, *77*(5), 1–35. https://doi.org/10.18637/jss.v077.i05

Driver, C. C., & Tomasik, M. J. (2023). Formalizing developmental phenomena as continuous-time systems: Rela-

tions between mathematics and language development. *Child Development*, *94*(6), 1454–1471. https://doi.org/10.1111/cdev.13990

Driver, C. C., & Voelkle, M. C. (2018a). Hierarchical Bayesian continuous time dynamic modeling. *Psychological Methods*, *23*(4), 774–799. https://doi.org/10.1037/met0000168

Driver, C. C., & Voelkle, M. C. (2018b). Understanding the time course of interventions with continuous time dynamic models. In K. van Montfort, J. H. L. Oud, & M. C. Voelkle (Eds.), *Continuous time modeling in the behavioral and related sciences* (pp. 79–109). Springer International Publishing. //www.springer.com/de/book/9783319772189

Driver, C. C., Voelkle, M. C., & Oud, J. H. L. (2025). *Ctsem: Continuous time structural equation modelling* [Manual]. CRAN. https://cran.r-project.org/package=ctsem

Durbin, J., & Koopman, S. J. (2012). *Time series analysis by state space methods* (2nd ed.). Oxford University Press.

Epskamp, S. (2020). Psychometric network models from time-series and panel data. *Psychometrika*, *85*(1), 206–231. https://doi.org/10.1007/s11336-020-09697-3

Gabry, J., Simpson, D., Vehtari, A., Betancourt, M., & Gelman, A. (2019). Visualization in bayesian workflow. *Journal of the Royal Statistical Society: Series A*, *182*(2), 389–402. https://doi.org/10.1111/rssa.12378

Gelman, A., Meng, X.-L., & Stern, H. (1996). Posterior predictive assessment of model fitness via realized discrepancies. *Statistica Sinica*, *6*(4), 733–760. https://www.jstor.org/stable/24306036

Gelman, A., & Shalizi, C. R. (2013). Philosophy and the practice of Bayesian statistics. *British Journal of Mathematical and Statistical Psychology*, *66*(1), 8–38. https://doi.org/10.1111/j.2044-8317.2011.02037.x

Haehner, P., Driver, C. C., Hopwood, C. J., Luhmann, M., Fliedner, K., & Bleidorn, W. (2025). The dynamics of self-esteem and depressive symptoms across days, months, and years. *Journal of Personality and Social Psychology*. https://doi.org/10.1037/pspp0000542

Hamaker, E. L., Kuiper, R. M., & Grasman, R. P. P. P. (2015). A critique of the cross-lagged panel model. *Psychological Methods*, *20*(1), 102–116. https://doi.org/10.1037/a0038889

Harvey, A. C. (1989). *Forecasting, structural time series models and the kalman filter*. Cambridge University Press.

Haslbeck, J. M. B., Ryan, O., Robinaugh, D. J., Waldorp, L. J., & Borsboom, D. (2022). Modeling psychopathology: From data models to formal theories. *Psychological Methods*, *27*(6), 930–957. https://doi.org/10.1037/met0000303

Higham, D. J. (2001). An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM Review*, *43*(3), 525–546. https://doi.org/10.1137/S0036144500378302

Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, *82*(1), 35–45. https://doi.org/10.1115/1.3662552

Kenny, D. A. (2005). Cross-Lagged Panel Design. In *Encyclopedia of Statistics in Behavioral Science*. John Wiley & Sons, Ltd. https://doi.org/10.1002/0470013192.bsa156

Kloeden, P. E., & Platen, E. (1992). *Numerical solution of stochastic differential equations*. Springer.

Kuiper, R. M., & Ryan, O. (2018). Drawing conclusions from cross-lagged relationships: Re-considering the role of the time-interval. *Structural Equation Modeling: A Multidisciplinary Journal*, *25*(5), 809–823. https://doi.org/10.1080/10705511.2018.1431046

Nakagawa, S., & Schielzeth, H. (2013). A general and simple method for obtaining R^2 from generalized linear mixed-effects models. *Methods in Ecology and Evolution*, *4*(2), 133–142. https://doi.org/10.1111/j.2041-210X.2012.00261.x

Oravecz, Z., Tuerlinckx, F., & Vandekerckhove, J. (2011). A hierarchical latent stochastic differential equation model for affective dynamics. *Psychological Methods*, *16*(4), 468–490. https://doi.org/10.1037/a0024375

Ou, L., Hunter, M. D., & Chow, S.-M. (2019). What's for dynr: A package for linear and nonlinear dynamic modeling in R. *The R Journal*, *11*(1), 91–111. https://doi.org/10.32614/rj-2019-012

Robinaugh, D. J., Haslbeck, J. M. B., Ryan, O., Fried, E. I., & Waldorp, L. J. (2021). Invisible hands and fine calipers: A call to use formal theory as a toolkit for theory construction. *Perspectives on Psychological Science*, *16*(4), 725–743. https://doi.org/10.1177/1745691620974697

Rohrer, J. M., & Murayama, K. (2023). These are not the effects you are looking for: Causality and the within-/between-persons distinction in longitudinal data analysis. *Advances in Methods and Practices in Psychological Science*, *6*(1), 25152459221140842. https://doi.org/10.1177/25152459221140842

Rosseel, Y., & Vidal, M. (2025). A tutorial for understanding SEM using R: Where do all the numbers come from? *British Journal of Mathematical and Statistical Psychology*. https://doi.org/10.1111/bmsp.70003

Ryan, O., & Hamaker, E. L. (2022). Time to intervene: A continuous-time approach to network analysis and centrality. *Psychometrika*, *87*(1), 214–252. https://doi.org/10.1007/s11336-021-09767-0

van Geert, P. (2011). The contribution of complex dynamic systems to development. *Child Development Perspectives*, *5*(4), 273–278. https://doi.org/10.1111/j.1750-8606.2011.00197.x

Voelkle, M. C., & Oud, J. H. L. (2013). Continuous time modelling with individually varying time intervals for os-

cillating and non-oscillating processes. *British Journal of Mathematical and Statistical Psychology*, *66*(1), 103–126. https://doi.org/10.1111/j.2044-8317.2012.02043.x

Voelkle, M. C., Oud, J. H. L., Davidov, E., & Schmidt, P. (2012). An SEM approach to continuous time modeling of panel data: Relating authoritarianism and anomia. *Psychological Methods*, *17*(2), 176–192. https://doi.org/10.1037/a0027543

Wickham, H. (2011). Ggplot2. *Wiley Interdisciplinary Reviews: Computational Statistics*, *3*(2), 180–185.

Wilson, R. C., & Collins, A. G. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife*, *8*, e49547. https://doi.org/10.7554/eLife.49547